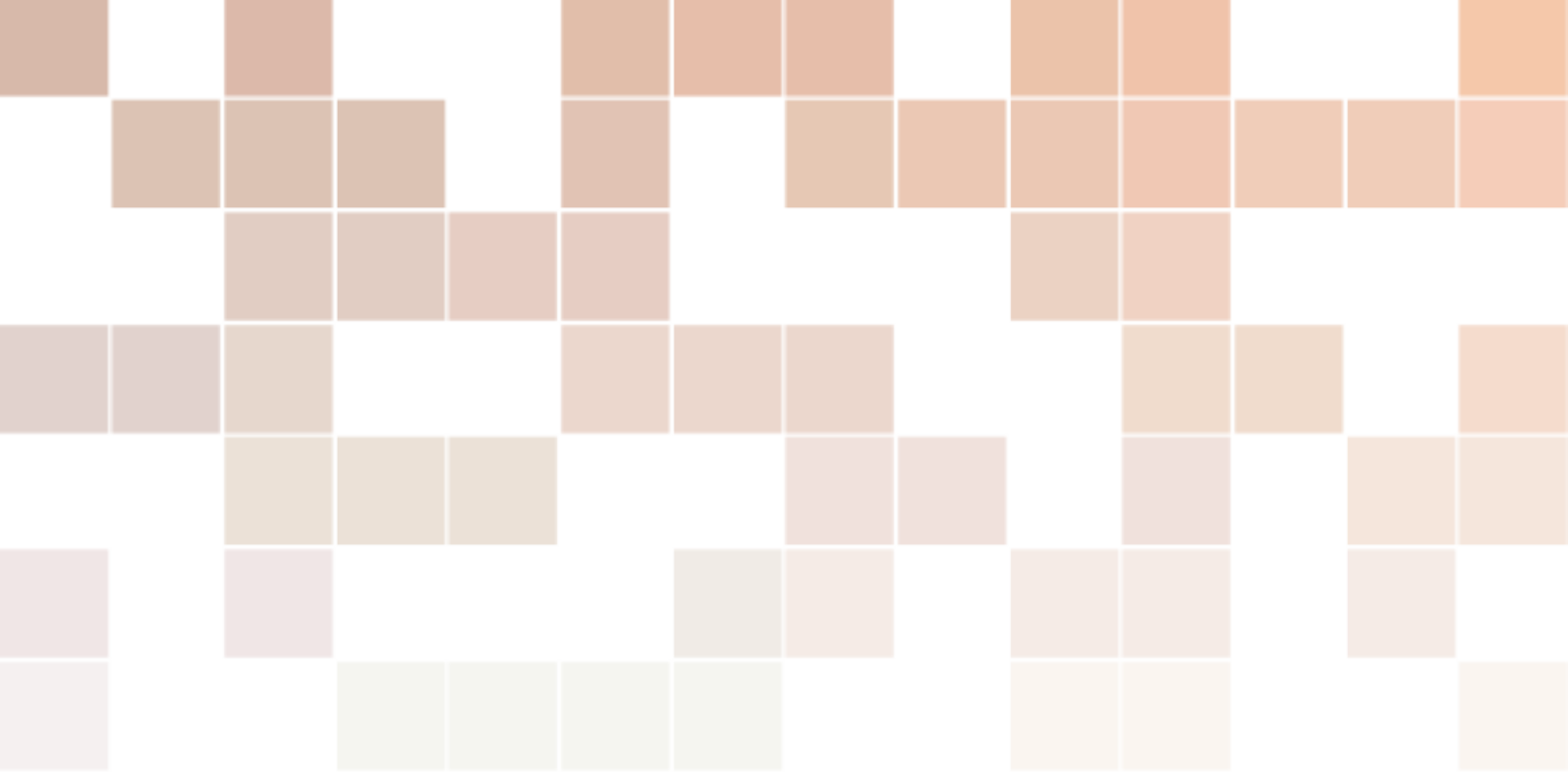
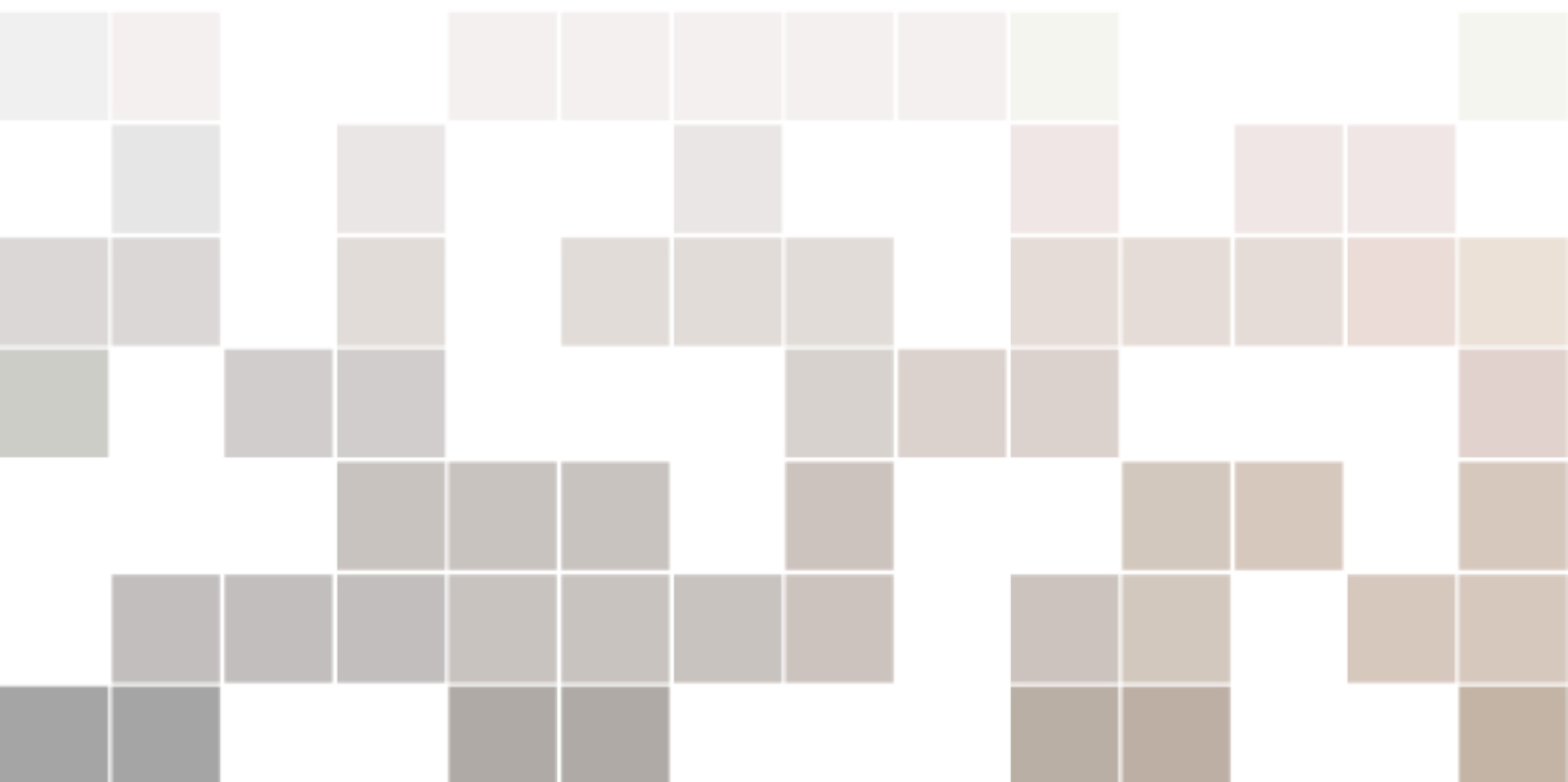




强化学习和多模态教程

理论与实践

左元



目录

III

多模态

1	Vision Transformer	15
1.1	导入库和模块	16
1.2	补丁嵌入 (Patch Embedding)	16
1.3	类别对应的 token 和位置编码	18
1.4	注意力头	20
1.5	多头注意力	22
1.6	Transformer 编码器	23
1.7	Vision Transformer	24
1.8	训练参数	26
1.9	加载 MNIST 数据集	26
1.10	训练循环	27
1.11	评估模型	27
2	CLIP	29
2.1	导入库和模块	29
2.2	图像和文本编码器	30
2.2.1	位置嵌入	30
2.2.2	注意力头	30
2.2.3	多头注意力	31
2.2.4	Transformer 编码器	31
2.2.5	文本的分词器	32
2.2.6	文本编码器	33
2.2.7	图像编码器	35
2.3	CLIP 模型	36
2.3.1	数据	39
2.3.2	训练参数	41

2.3.3	加载数据集	42
2.3.4	训练模型	42
2.3.5	评估模型	43
2.3.6	零样本分类	44
3	ClipCap (图生文)	47
3.1	ClipCap 的工作原理	47
3.2	关于训练	48
3.3	推理	48
3.4	ClipCap 的代码实现	49
4	扩散模型	51
4.1	通俗讲解	51
4.1.1	扩散	51
4.1.2	扩散模型的架构	52
4.1.3	模型设计	54
4.2	扩散模型理论简介	56
4.2.1	概述	56
4.2.2	正向扩散过程	57
4.2.3	反向扩散过程	58
4.2.4	U-Net 模型	58
4.3	代码实现	63
4.3.1	扩散过程相关代码	63
4.3.2	时间步位置编码	65
4.3.3	U-Net 神经网络	66
4.4	完整代码	67
5	条件扩散模型	75
5.1	向扩散模型添加条件	75
5.2	条件扩散模型的实现	77
5.3	得分函数	79
5.3.1	什么是得分函数	80
5.3.2	式 (5.5) 的证明	81
5.4	分类器指引	82
5.4.1	什么是分类器	82
5.4.2	分类器指引的推导	82
5.5	无分类器指引	84
5.5.1	无分类器指引的理论知识	84
5.5.2	无分类器指引的实现	85
5.6	Stable Diffusion	87

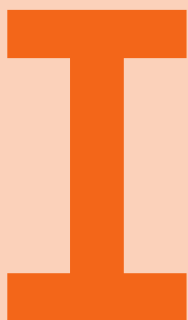
List of Figures

1.1	Vit 架构图	15
1.2	补丁的例子	16
1.3	将图像转换为补丁: 第一步	17
1.4	将图像转换为补丁: 第二步	18
1.5	将图像转换为补丁: 第三步	18
1.6	位置编码的作用	19
1.7	左图: 使用 Conv2D 运算分成 16 个 8x8 块的 32x32 MNIST 图像。右图: 添加位置编码和类别 token 后的 16 个图像补丁, 使用随机数据初始化。	20
1.8	注意力机制	21
2.1	clip 原理, 来自原始论文	29
2.2	注意力分数的掩码	31
2.3	分词过程	33
2.4	余弦相似度的计算	38
2.5	复制掩码	40
2.6	因果注意力分数的掩码	41
3.1	图片标题模型的简单示例	47
3.2	将图片的 CLIP 嵌入映射到 GPT2 的嵌入空间	48
3.3	ClipCap 的推理过程	49
4.1	水滴的扩散过程	51
4.2	高清图像的扩散过程	52
4.3	从高斯分布中采样一个随机值	53
4.4	预测图像与实际图像之间的差异使用损失函数计算	53
4.5	将噪声作为预测目标	54
4.6	U-net 架构 (以最低分辨率为 32x32 像素为例)。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 x-y 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。	55
4.7	共享模型	56
4.8	正向扩散和逆向扩散	56
4.9	正向扩散公式图解	57
4.10	以噪声为预测目标	58
4.11	训练 unet	59
4.12	U-net 训练算法	59

4.13	扩散模型训练步骤	60
4.14	采样算法（去噪算法，反向扩散算法）	61
4.15	扩散模型的采样过程	62
4.16	以噪声为预测目标	64
4.17	采样算法（去噪算法，反向扩散算法）	65
4.18	U-net 模型	66
5.1	本章将要实现的条件扩散模型	75
5.2	预测时刻 $t - 1$ 的图像（正态分布的均值向量）的模型（上）和预测添加到 x_t 中的噪声的模型（下）	76
5.3	条件扩散模型的神经网络	77
5.4	在扩散模型中添加嵌入层	77
5.5	扩散模型中使用的推断噪声 ε 的神经网络	80
5.6	用于推断得分的神经网络	80
5.7	使用预训练神经网络进行分类的示例（参数为 ϕ ）	82
5.8	通过反向传播可以求出 $\nabla_{x_t} \log p(y x_t)$	83
5.9	使用了推断得分的神经网络的分类器指引	84
5.10	上面式子的可视化	85
5.11	Stable Diffusion 生成的高清图像	88



List of Tables



强化学习

II 基于人类反馈的强化学习



多模态

1	Vision Transformer	15
1.1	导入库和模块	16
1.2	补丁嵌入 (Patch Embedding)	16
1.3	类别对应的 token 和位置编码	18
1.4	注意力头	20
1.5	多头注意力	22
1.6	Transformer 编码器	23
1.7	Vision Transformer	24
1.8	训练参数	26
1.9	加载 MNIST 数据集	26
1.10	训练循环	27
1.11	评估模型	27
2	CLIP	29
2.1	导入库和模块	29
2.2	图像和文本编码器	30
2.3	CLIP 模型	36
3	ClipCap (图生文)	47
3.1	ClipCap 的工作原理	47
3.2	关于训练	48
3.3	推理	48
3.4	ClipCap 的代码实现	49
4	扩散模型	51
4.1	通俗讲解	51
4.2	扩散模型理论简介	56
4.3	代码实现	63
4.4	完整代码	67
5	条件扩散模型	75
5.1	向扩散模型添加条件	75
5.2	条件扩散模型的实现	77
5.3	得分函数	79
5.4	分类器指引	82
5.5	无分类器指引	84
5.6	Stable Diffusion	87

1. Vision Transformer

🔥 提示

ViT: Vision Transformer

基于自注意力的 Transformer 模型由 Vaswani 等人在 2017 年的论文《Attention Is All You Need》中首次提出，并已广泛应用于自然语言处理中。Transformer 模型是 OpenAI 用来创建 ChatGPT 的模型。Transformer 不仅适用于文本，还适用于图像，基本上可以处理任何序列数据。2021 年，Dosovitsky 等人在他们的论文《An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale》中引入了将 Transformer 用于计算机视觉任务（例如图像分类）的想法。在他们的论文中，与卷积网络相比，他们的 Vision Transformer 模型能够取得出色的结果，并且需要更少的资源来训练。

在本教程中，我们将从头开始构建一个 Vision Transformer 模型，并在 MNIST 数据集上进行测试。

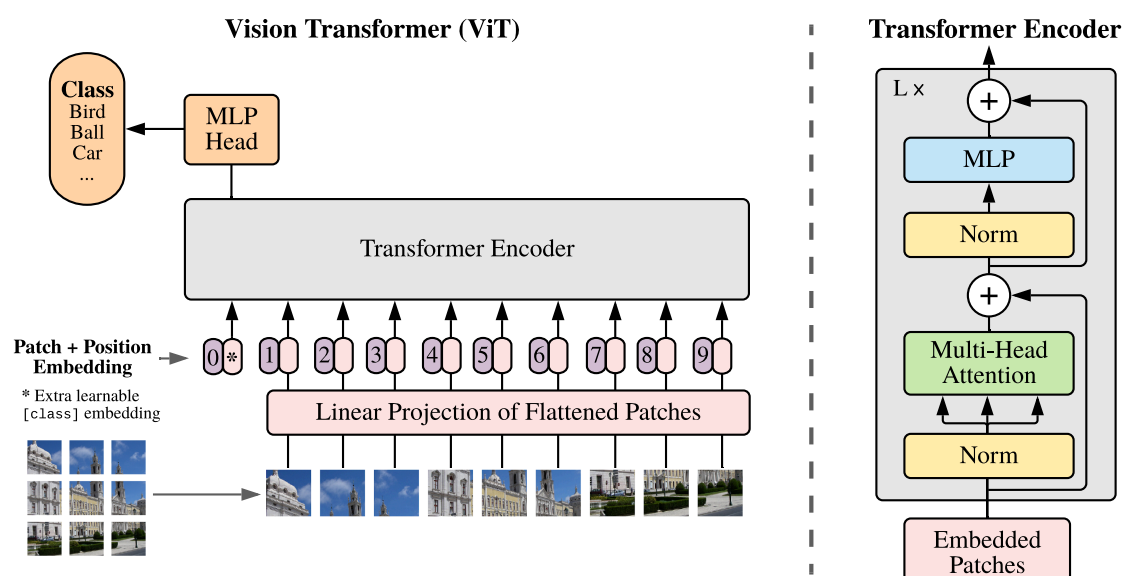


图 1.1 Vit 架构图

1.1 导入库和模块

本章所需依赖

Python

```

1 import torch
2 import torch.nn as nn
3 import torchvision.transforms as T
4 from torch.optim import Adam
5 from torchvision.datasets.mnist import MNIST
6 from torch.utils.data import DataLoader
7 import numpy as np

```

1.2 补丁嵌入 (Patch Embedding)

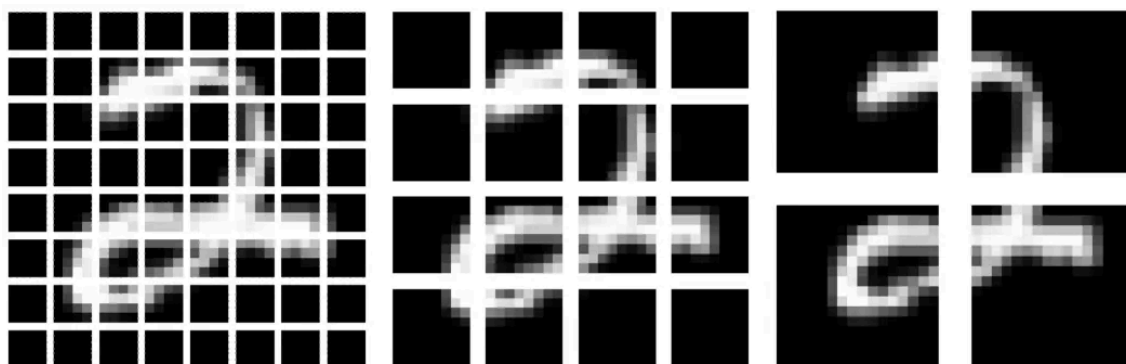


图 1.2 补丁的例子

补丁嵌入类

Python

```

1 class PatchEmbedding(nn.Module):
2     def __init__(
3         self,
4         d_model,    # 模型的维度
5         img_size,   # 图片大小
6         patch_size, # 补丁大小
7         n_channels  # 通道数量
8     ):
9         super().__init__()
10
11         self.d_model = d_model
12         self.img_size = img_size
13         self.patch_size = patch_size
14         self.n_channels = n_channels
15
16         self.linear_project = nn.Conv2d(
17             self.n_channels, # in_channels
18             self.d_model, # out_channels
19             kernel_size=self.patch_size, # kernel_size
20             stride=self.patch_size # stride
21         )

```

```

22
23     # B: 批次大小
24     # C: 通道数量
25     # H: 图像高度
26     # W: 图像宽度
27     # P_col: 补丁的列
28     # P_row: 补丁的行
29     def forward(self, x):
30         x = self.linear_project(x) # (B, C, H, W) → (B, d_model, P_col, P_row)
31         x = x.flatten(2) # (B, d_model, P_col, P_row) → (B, d_model, P)
32         x = x.transpose(1, 2) # (B, d_model, P) → (B, P, d_model)
33         return x

```

创建 Vision Transformer 的第一步是将输入图像拆分为补丁，并创建这些补丁的线性嵌入序列。我们能够通过使用 PyTorch 的 Conv2d 方法来实现这一点。

Conv2d 方法获取输入图像，将它们拆分为补丁，并提供大小等于 `d_model` 的线性投影。通过将 `kernel_size` 和步幅设置为补丁大小，我们确保补丁大小正确且没有重叠。

```

1 self.linear_project = nn.Conv2d(
2     self.n_channels,
3     self.d_model,
4     kernel_size=self.patch_size,
5     stride=self.patch_size
6 )

```

[Python](#)

在 `forward` 方法中，我们通过 `linear_project/Conv2D` 方法传递具有形状 (B, C, H, W) 的输入，并输出形状 $(B, d_model, P_col, P_row)$ 的张量。

```

1 def forward(self, x):
2     x = self.linear_project(x) # (B, C, H, W) → (B, d_model, P_col, P_row)

```

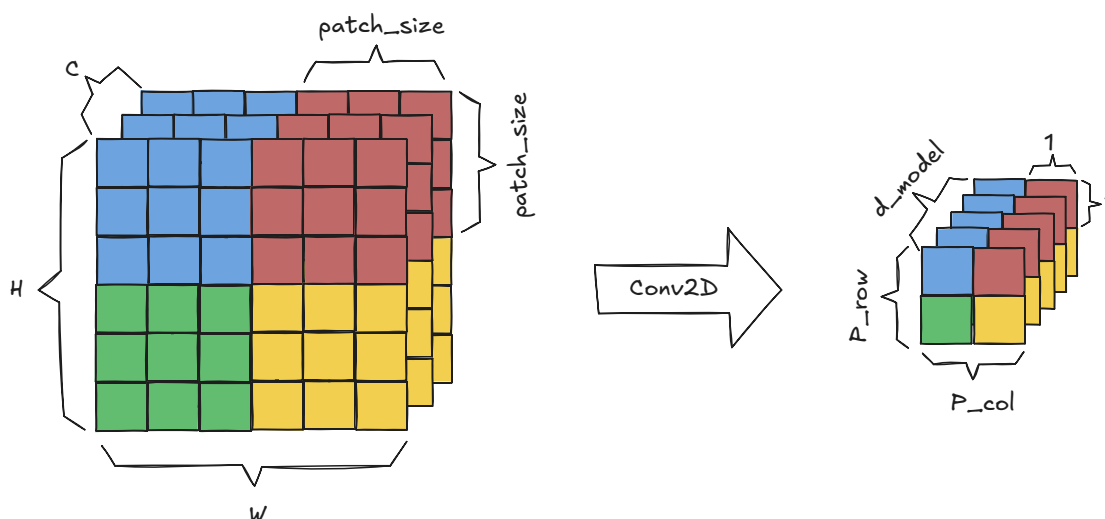
[Python](#)


图 1.3 将图像转换为补丁：第一步

我们使用展平方法将补丁列和补丁行维度组合成一个补丁维度，从而得到 (B, d_model, P) 的形状

```

1 x = x.flatten(2) # (B, d_model, P_col, P_row) → (B, d_model, P)

```

[Python](#)

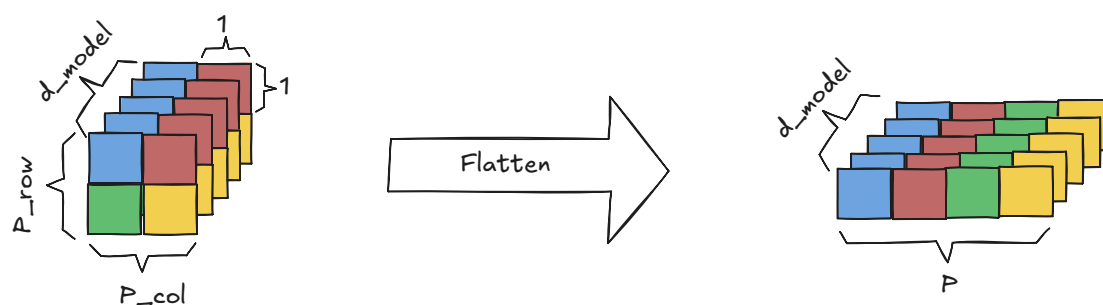


图 1.4 将图像转换为补丁：第二步

最后，我们使用转置方法切换 d_model 和补丁维度，得到 (B, P, d_model) 的形状。

```
1 x = x.transpose(1, 2) # (B, d_model, P) → (B, P, d_model)
```

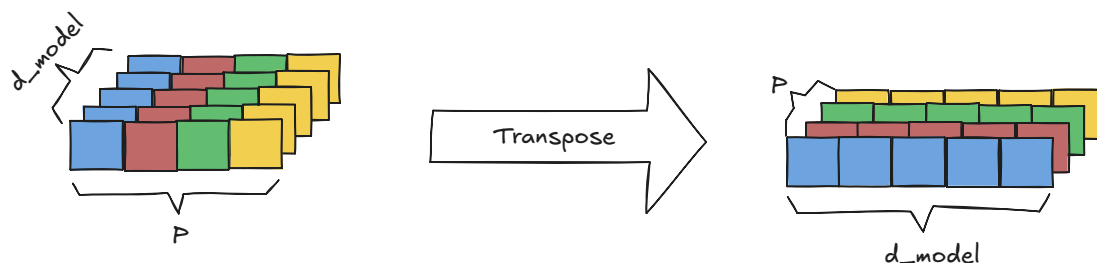
[Python](#)


图 1.5 将图像转换为补丁：第三步

1.3 类别对应的 token 和位置编码

位置编码类

[Python](#)

```
1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model, max_seq_length):
3         super().__init__()
4         # 类别token
5         self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))
6         # 创建位置编码
7         pe = torch.zeros(max_seq_length, d_model)
8
9         for pos in range(max_seq_length):
10             for i in range(d_model):
11                 if i % 2 == 0:
12                     pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
13                 else:
14                     pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))
15         # 将位置编码固定
16         self.register_buffer("pe", pe.unsqueeze(0))
17
18     def forward(self, x):
19         # 为批次中的每张图片分配一个类别token
```

```

20 tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
21 # 将类别token添加到每个图像的补丁嵌入数组的开头
22 x = torch.cat((tokens_batch,x), dim=1)
23 # 将位置编码添加到嵌入中
24 x = x + self.pe
25 return x

```

ViT 模型使用向补丁嵌入添加可学习的类别 token 的标准方法来执行分类。

```

1 # class token: 类别或者分类对应的token
2 self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))

```

Python

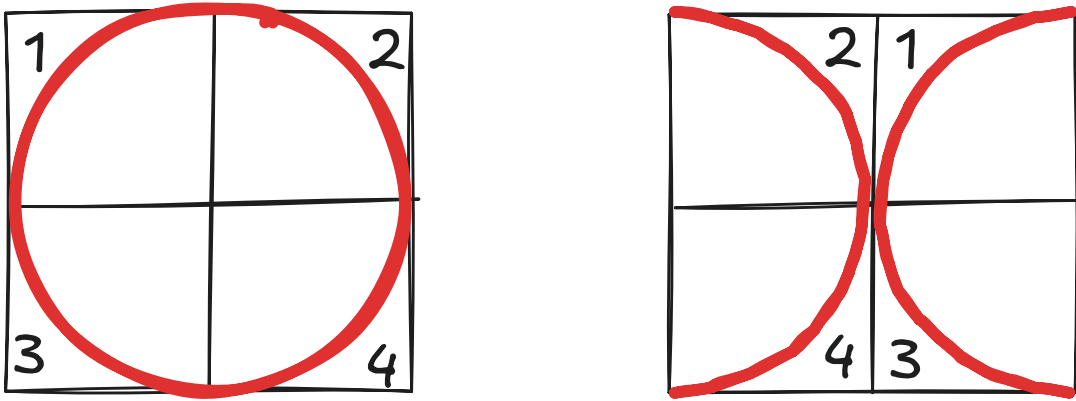


图 1.6 位置编码的作用

与 LSTM 等按顺序接受嵌入的模型不同，Transformer 并行的接受嵌入。虽然这提高了速度，但 Transformer 不知道序列的顺序是什么。这是一个很大的问题，因为改变序列的顺序很可能会改变其含义。上图就是一个例子，它显示更改图像的补丁顺序可以将图像从 0 更改为更接近 X 的图像。为了解决这个问题，需要将位置编码添加到嵌入中。每个位置编码对于它所表示的位置都是唯一的，这允许模型识别每个补丁嵌入应该在的位置。为了将位置编码添加到补丁嵌入中，它们必须具有相同的维度， d_{model} 。我们使用下面的公式获得位置编码。

Definition 1.1 — 位置编码。

$$\begin{aligned}
 \text{PE}_{(\text{pos}, 2i)} &= \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \\
 \text{PE}_{(\text{pos}, 2i+1)} &= \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)
 \end{aligned}
 \tag{1.1}$$

```

1 # max_seq_length: 补丁嵌入的数量, d_model: 嵌入维度
2 pe = torch.zeros(max_seq_length, d_model)
3
4 for pos in range(max_seq_length):
5     for i in range(d_model):
6         if i % 2 == 0:
7             pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
8         else:
9             pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))

```

Python


```
10 # 禁止pe更新, pe保存在了显存中, 不会被反向传播更新
11 self.register_buffer('pe', pe.unsqueeze(0))
```

在 `forward` 方法中, 输入是多个图像的一批补丁嵌入。例如, 32×32 的图像可以分解为 16 个 8×8 大小的补丁。在此 `max_seq_length` 中, 需要为 $16+1=17$ 才能创建足够的位置嵌入, 每个补丁一个, 分类对应的 `token` 一个。

因此, 我们需要使用 `expand` 函数才能使用 `self.cls_token` 为批处理中的每个图像创建分类对应的 `token`。注意力机制会将有关整个序列的信息编码到序列中的每个 `token` 中。由于每个 `token` 都受到其自身信息的偏见, 因此分类对应的 `token` 会创建序列中所有 `token` 的独立的摘要信息。

```
1 def forward(self, x):
2     tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
```

[Python](#)

然后, 使用 `torch.cat` 方法将这些分类 `token` 添加到每个补丁嵌入的开头。

```
1 x = torch.cat((tokens_batch, x), dim=1)
```

[Python](#)

位置编码在输出之前添加。

```
1 x = x + self.pe
2 return x
```

[Python](#)

这是添加分类 `token` 之前和之后的数据的样子:

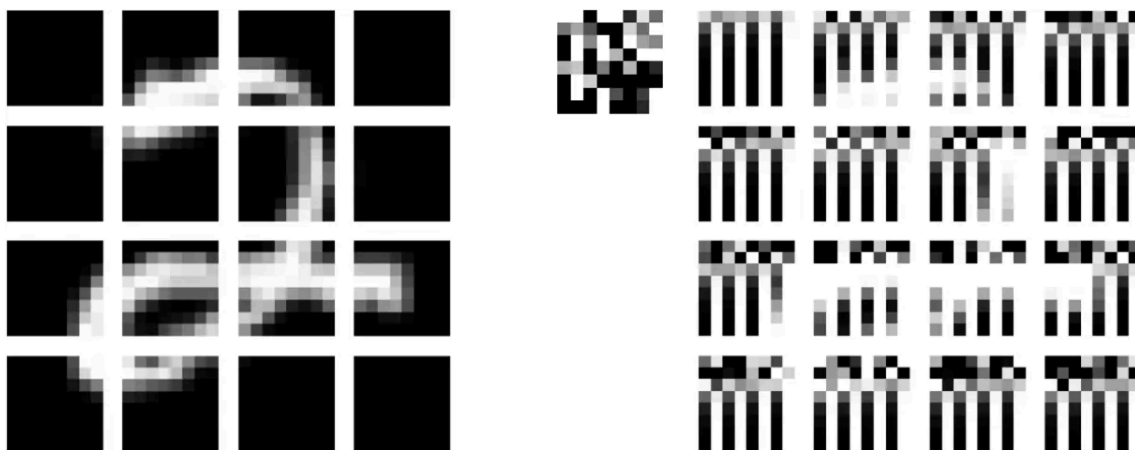


图 1.7 左图: 使用 `Conv2D` 运算分成 16 个 8×8 块的 32×32 MNIST 图像。右图: 添加位置编码和类别 `token` 后的 16 个图像补丁, 使用随机数据初始化。

请注意, 我们已经用 64 个卷积核初始化了 `Conv2D` 运算, 每个卷积核中的每个补丁只占用一个像素, 以免扭曲图像。

1.4 注意力头

注意力头

[Python](#)

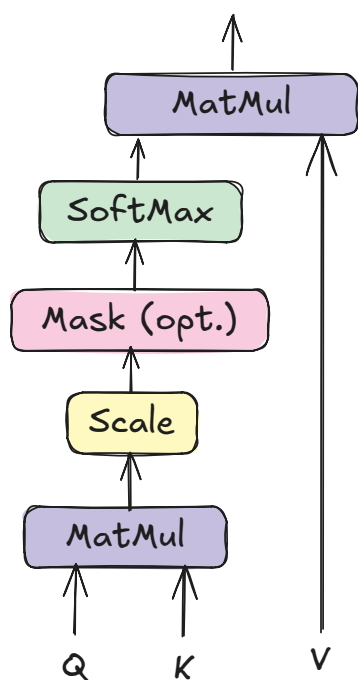
```
1 class AttentionHead(nn.Module):
2     def __init__(self, d_model, head_size):
3         super().__init__()
4         self.head_size = head_size
5
6         self.query = nn.Linear(d_model, head_size)
```

```

7     self.key = nn.Linear(d_model, head_size)
8     self.value = nn.Linear(d_model, head_size)
9
10    def forward(self, x):
11        # 计算Q, K, V
12        Q = self.query(x)
13        K = self.key(x)
14        V = self.value(x)
15
16        #  $QK^T$ 
17        attention = Q @ K.transpose(-2, -1)
18
19        #  $\text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{head}}}}\right)V$ 
20        attention = attention / (self.head_size ** 0.5)
21        attention = torch.softmax(attention, dim=-1)
22        attention = attention @ V
23
24    return attention

```

Scaled Dot-Product Attention



Multi-Head Attention

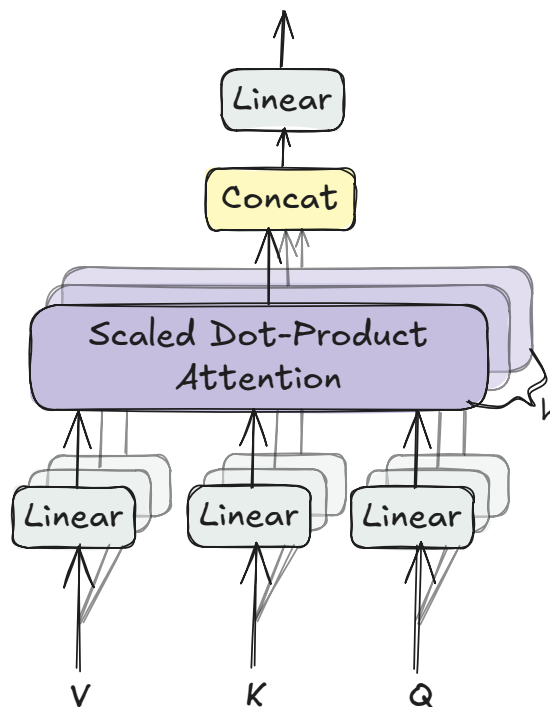


图 1.8 注意力机制

ViT 使用注意力机制，这是一种通信机制，允许模型专注于图像的重要部分。可以使用下面的公式计算注意力分数。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.2)$$

计算注意力的第一步是获取 token 的 Q、K 和 V。token 的 Q 是 token 要查找的内容，K 是 token 包含的内容，V 是 token 之间的互信息。Q、K 和 V 可以通过线性层传递 token 来计算。

```
1 def forward(self, x):
2     # 计算Q, K, V
3     Q = self.query(x)
4     K = self.key(x)
5     V = self.value(x)
```

 Python

我们能够通过计算 Q 和 K 的点积来获取序列中 token 之间的关系。

```
1 attention = Q @ K.transpose(-2, -1)
```

 Python

我们需要缩放这些值以控制初始化时的方差,以便 token 能够聚合来自多个其他 token 的信息。通过将点积除以注意力头大小的平方根来应用缩放。

```
1 attention = attention / (self.head_size ** 0.5)
```

 Python

然后,我们需要对缩放的点积应用 softmax。

```
1 attention = torch.softmax(attention, dim=-1)
```

 Python

最后,我们需要得到 softmax 和 V 矩阵之间的点积。这本质上是在相应的 token 之间传递信息。

```
1 attention = attention @ V
2 return attention
```

 Python

1.5 多头注意力

多头注意力

 Python

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, n_heads):
3         super().__init__()
4         self.head_size = d_model // n_heads
5         self.W_o = nn.Linear(d_model, d_model)
6         self.heads = nn.ModuleList([
7             AttentionHead(d_model, self.head_size) for _ in range(n_heads)
8         ])
9
10    def forward(self, x):
11        # 拼接多个注意力头
12        out = torch.cat([head(x) for head in self.heads], dim=-1)
13        out = self.W_o(out)
14        return out
```

多头注意力只是并行运行多个头的自注意力并将它们组合起来。我们可以通过将注意力头添加到模块列表中来做到这一点,

```
1 self.heads = nn.ModuleList([
2     AttentionHead(d_model, self.head_size)
3     for _ in range(n_heads)
4 ])
```

 Python

传递输入然后拼接。

```
1 def forward(self, x):
```

 Python

```

2     # 拼接多个注意力头
3     out = torch.cat([head(x) for head in self.heads], dim=-1)

```

然后，我们需要将输出传递给另一个线性层。

```

1 out = self.W_o(out)
2 return out

```

 Python

1.6 Transformer 编码器

Transformer编码器

 Python

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, d_model, n_heads, r_mlp=4):
3         super().__init__()
4         self.d_model = d_model
5         self.n_heads = n_heads
6
7         # 层归一化
8         self.ln1 = nn.LayerNorm(d_model)
9
10        # 多头注意力
11        self.mha = MultiHeadAttention(d_model, n_heads)
12
13        # 层归一化
14        self.ln2 = nn.LayerNorm(d_model)
15
16        # MLP
17        self.mlp = nn.Sequential(
18            nn.Linear(d_model, d_model*r_mlp),
19            nn.GELU(),
20            nn.Linear(d_model*r_mlp, d_model)
21        )
22
23    def forward(self, x):
24        out = x + self.mha(self.ln1(x)) # 跳跃连接
25        out = out + self.mlp(self.ln2(out)) # 跳跃连接
26        return out

```

Transformer 编码器由两个子层组成：第一个子层执行多头注意力，第二个子层包含 MLP。多头注意力子层执行 token 之间的通信，而 MLP 子层允许 token 单独“思考”与它们通信的内容。

层归一化是一种优化技术，可跨其特征独立归一化批处理中的每个输入。对于我们的模型，我们将在每个子层的开头通过层归一化模块传递我们的输入。

```

1 self.ln1 = nn.LayerNorm(d_model)
2 self.ln2 = nn.LayerNorm(d_model)

```

 Python

MLP 将由两个线性层组成，中间有一个 GELU 层。使用 GELU 代替 RELU，因为它没有 RELU 在零点不可导的限制。

```

1 # 编码器的MLP
2 self.mlp = nn.Sequential(
3     nn.Linear(width, width*r_mlp),

```

 Python

```

4     nn.GELU(),
5     nn.Linear(width*r_mlp, width)
6 )

```

在编码器的 `forward` 方法中，输入在执行多头注意力之前通过第一层归一化模块。通过执行多头注意力将原始输入添加到输出中，以创建残差连接。

然后，在输入 MLP 之前，它会通过另一个层归一化模块。通过将 MLP 的输出添加到第一个残差连接的输出，创建另一个残差连接。

残差连接用于通过创建一条路径来帮助防止梯度消失问题，以便梯度不受阻碍地反向传播回原始输入。

```

1 def forward(self, x):
2     out = x + self.mha(self.ln1(x))
3     out = out + self.mlp(self.ln2(out))
4     return out

```

 Python

1.7 Vision Transformer

```

ViT类
1 class VisionTransformer(nn.Module):
2     def __init__(
3         self,
4         d_model,
5         n_classes,
6         img_size,
7         patch_size,
8         n_channels,
9         n_heads,
10        n_layers
11    ):
12        super().__init__()
13
14        assert img_size[0] % patch_size[0] == 0 \
15            and img_size[1] % patch_size[1] == 0, \
16            "img_size 必须能被 patch_size 整除"
17        assert d_model % n_heads == 0, \
18            "d_model 必须能被 n_heads 整除"
19
20        self.d_model = d_model # 模型维度，嵌入的维度（宽度）
21        self.n_classes = n_classes # 类别的数量
22        self.img_size = img_size # 图片大小
23        self.patch_size = patch_size # 补丁大小
24        self.n_channels = n_channels # 通道数
25        self.n_heads = n_heads # 注意力头的数量
26        # 补丁的数量 =  $\frac{32 \times 32}{4 \times 4}$ 
27        self.n_patches = (self.img_size[0] * self.img_size[1]) \
28            // (self.patch_size[0] * self.patch_size[1])
29        # 序列的长度 = 1（分类token） + 补丁的数量
30        self.max_seq_length = self.n_patches + 1
31        # 补丁嵌入

```

 Python

```

32     self.patch_embedding = PatchEmbedding(
33         self.d_model,
34         self.img_size,
35         self.patch_size,
36         self.n_channels
37     )
38     # 位置编码
39     self.positional_encoding = PositionalEncoding(
40         self.d_model,
41         self.max_seq_length
42     )
43     self.transformer_encoder = nn.Sequential(*[
44         TransformerEncoder(self.d_model, self.n_heads)
45         for _ in range(n_layers)
46     ])
47
48     # 用于分类的MLP
49     self.classifier = nn.Sequential(
50         nn.Linear(self.d_model, self.n_classes),
51         nn.Softmax(dim=-1)
52     )
53
54     def forward(self, images):
55         # 将图片转换成补丁的嵌入 (embedding)
56         x = self.patch_embedding(images)
57         # 添加位置编码
58         x = self.positional_encoding(x)
59         # 编码
60         x = self.transformer_encoder(x)
61         # 分类的线性层
62         x = self.classifier(x[:,0])
63         return x

```

在创建我们的 `VisionTransformer` 类时，我们首先需要确保输入图像可以均匀地分成大小为 `patch_size` 的补丁，并且模型的维数可以被注意力头的数量整除。

```

1  assert img_size[0] % patch_size[0] == 0 \
2      and img_size[1] % patch_size[1] == 0, \
3      "img_size 必须能被 patch_size 整除"
4  assert d_model % n_heads == 0, \
5      "d_model 必须能被 n_heads 整除"

```

 Python

我们还需要计算位置编码的最大序列长度，该长度等于补丁的数量加 1。补丁的数量可以通过将输入图像的高和宽的乘积除以补丁大小的高和宽的乘积来计算。

```

1  self.n_patches = (self.img_size[0] * self.img_size[1]) \
2      // (self.patch_size[0] * self.patch_size[1])
3  self.max_seq_length = self.n_patches + 1

```

 Python

`VisionTransformer` 还需要能够拥有多个编码器模块。这可以通过将编码器层列表放入顺序包装器中来实现。

```
1 self.encoder = nn.Sequential(*[
2     TransformerEncoder(self.d_model, self.n_heads) for _ in range(n_layers)
3 ])
```

Python

VisionTransformer 模型的最后一部分是 MLP 分类头。它由一个线性层和一个 softmax 层组成。

```
1 self.classifier = nn.Sequential(
2     nn.Linear(self.d_model, self.n_classes),
3     nn.Softmax(dim=-1)
4 )
```

Python

在 forward 方法中，输入图像首先经过补丁嵌入层，将图像分割成补丁，并获取这些补丁的线性嵌入序列。然后，它们经过位置编码层，添加分类 token 和位置编码，最后经过编码器模块。之后，分类 token 再经过分类 MLP，以确定图像的分类。

```
1 def forward(self, images):
2     x = self.patch_embedding(images)
3     x = self.position_embedding(x)
4     x = self.encoder(x)
5     x = self.classifier(x[:,0])
6     return x
```

Python

我们已经完成了模型的构建。现在我们需要对其进行训练和测试。

1.8 训练参数

```
1 d_model = 9 # 嵌入的维度9
2 n_classes = 10 # 类别数量为10
3 img_size = (32,32) # 图片大小为32x32
4 patch_size = (16,16) # 补丁的大小是16x16
5 n_channels = 1 # 灰度图片通道数量为1
6 n_heads = 3 # 3个注意力头
7 n_layers = 3 # 3层编码器
8 batch_size = 128 # 每个批次128张图片
9 epochs = 5 # 训练5个epoch
10 alpha = 0.005 # 学习率5e-3
```

Python

1.9 加载 MNIST 数据集

```
1 transform = T.Compose([
2     T.Resize(img_size), # 28x28 → 32x32
3     T.ToTensor() # 转换成torch.tensor
4 ])
5
6 train_set = MNIST(
7     root="datasets", train=True, download=True, transform=transform
8 )
9 test_set = MNIST(
10    root="datasets", train=False, download=True, transform=transform
11 )
12
```

Python

```
13 train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
14 test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

1.10 训练循环

下面的代码使用 MNIST 训练集训练我们的 VisionTransformer 类，并展示了整个 epoch 的平均损失。

```
1  device = torch.device("cuda")
2
3  ViT = VisionTransformer(
4      d_model,
5      n_classes,
6      img_size,
7      patch_size,
8      n_channels,
9      n_heads,
10     n_layers
11 ).to(device)
12
13 optimizer = Adam(ViT.parameters(), lr=alpha)
14 criterion = nn.CrossEntropyLoss()
15
16 for epoch in range(epochs):
17     training_loss = 0.0
18     for i, data in enumerate(train_loader, 0):
19         # 取出图像和对应的标签
20         inputs, labels = data
21         inputs, labels = inputs.to(device), labels.to(device)
22
23         optimizer.zero_grad()
24         outputs = ViT(inputs)
25         loss = criterion(outputs, labels)
26         loss.backward()
27         optimizer.step()
28
29         training_loss += loss.item()
30
31     print(f"Epoch {epoch + 1}/{epochs} loss: \
32         {training_loss / len(train_loader) :.3f}")
```

1.11 评估模型

```
1  correct = 0
2  total = 0
3
4  with torch.no_grad():
5      for data in test_loader:
6          images, labels = data
7          images, labels = images.to(device), labels.to(device)
```



```
8     outputs = ViT(images)
9     _, predicted = torch.max(outputs.data, 1)
10    total += labels.size(0)
11    correct += (predicted == labels).sum().item()
12    print(f"\n预测准确率: {100 * correct // total} %")
```

使用此模型，我们仅用 5 个 **epoch** 就能够在 MNIST 数据集上实现约 92% 的准确率。此示例展示了自注意力机制可以替代深度卷积网络。

2. CLIP

计算机视觉系统历来仅限于一组固定的类别，CLIP 是一场革命，它允许通过“预测哪些图像和文本配对在一起”来识别开放世界中的对象。CLIP 能够通过学习批量训练数据的图像和文本特征之间的余弦相似度来预测这一点。这在下图的对比预训练部分显示，其中图像之间的点积特征 $\{I_1, I_2, \dots, I_N\}$ 和文本特征 $\{T_1, T_2, \dots, T_N\}$ 被占用。

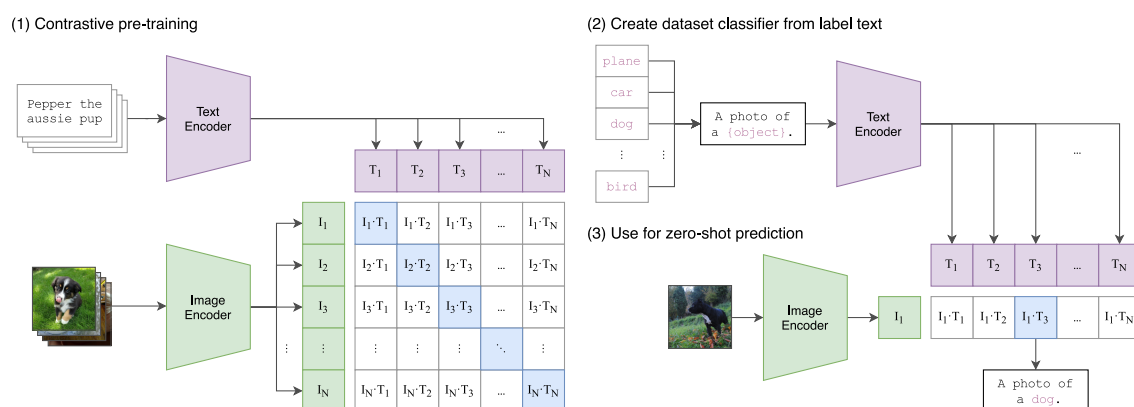


图 2.1 clip 原理，来自原始论文

在本章中，我们将从头开始构建 CLIP 并在 MNIST 数据集上对其进行测试。

2.1 导入库和模块

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision.transforms as T
5 from torch.utils.data import Dataset, DataLoader
6 from datasets import load_dataset
7 import matplotlib.pyplot as plt
8 import numpy as np
```

Python

2.2 图像和文本编码器

我们将首先构建图像和文本编码器。两者分别将图像和文本嵌入到单个 token 中,然后可用于对比损失计算。

2.2.1 位置嵌入

```
1 class PositionalEmbedding(nn.Module):
2     def __init__(self, width, max_seq_length):
3         super().__init__()
4         # 创建一个 (token的数量, 嵌入的维度) 形状的全0张量
5         # width 就是 d_model
6         pe = torch.zeros(max_seq_length, width)
7         # 将位置编码信息填充到pe中
8         for pos in range(max_seq_length):
9             for i in range(width):
10                 if i % 2 == 0:
11                     pe[pos][i] = np.sin(pos/(10000 ** (i/width)))
12                 else:
13                     pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/width)))
14         # 位置编码信息进行冻结, 不参与反向传播
15         self.register_buffer('pe', pe.unsqueeze(0))
16
17     def forward(self, x):
18         # 直接将位置编码和token的嵌入进行相加
19         x = x + self.pe
20         return x
```

2.2.2 注意力头

```
1 class AttentionHead(nn.Module):
2     def __init__(self, width, head_size):
3         super().__init__()
4         self.head_size = head_size
5
6         self.query = nn.Linear(width, head_size)
7         self.key = nn.Linear(width, head_size)
8         self.value = nn.Linear(width, head_size)
9
10    def forward(self, x, mask=None):
11        Q = self.query(x)
12        K = self.key(x)
13        V = self.value(x)
14
15        attention = Q @ K.transpose(-2, -1)
16        attention = attention / (self.head_size ** 0.5)
17        # 掩码
18        if mask is not None:
19            attention = attention.masked_fill(mask == 0, float("-inf"))
20        attention = torch.softmax(attention, dim=-1)
```

```

21     attention = attention @ V
22     return attention

```

Transformer 编码器和解码器之间的主要区别在于解码器使用注意力掩码，而编码器则不使用。虽然 CLIP 是仅编码器模型 (Encoder-Only)，但由于在分词时，对输入文本添加了 pad (填充符)，所以仍然需要与文本编码器一起使用掩码。请注意，掩码是可选的，因此这个注意力头可用于文本和视觉编码器。

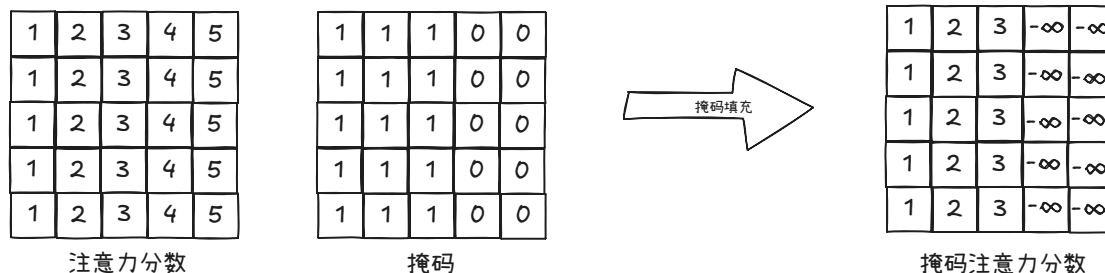


图 2.2 注意力分数的掩码

```

1 # 使用注意力掩码
2 if mask is not None:
3     attention = attention.masked_fill(mask == 0, float("-inf"))

```

Python

2.2.3 多头注意力

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, width, n_heads):
3         super().__init__()
4         self.head_size = width // n_heads
5         self.W_o = nn.Linear(width, width)
6         self.heads = nn.ModuleList([
7             AttentionHead(width, self.head_size)
8             for _ in range(n_heads)
9         ])
10
11     def forward(self, x, mask=None):
12         # 拼接多个注意力头
13         out = torch.cat([head(x, mask=mask) for head in self.heads], dim=-1)
14         out = self.W_o(out)
15         return out

```

Python

2.2.4 Transformer 编码器

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, width, n_heads, r_mlp=4):
3         super().__init__()
4         self.width = width # 嵌入的维度 d_model
5         self.n_heads = n_heads
6
7         # 层归一化

```

Python

```

8         self.ln1 = nn.LayerNorm(width)
9
10        # 多头注意力
11        self.mha = MultiHeadAttention(width, n_heads)
12
13        # 层归一化
14        self.ln2 = nn.LayerNorm(width)
15
16        # MLP
17        self.mlp = nn.Sequential(
18            nn.Linear(self.width, self.width*r_mlp),
19            nn.GELU(),
20            nn.Linear(self.width*r_mlp, self.width)
21        )
22
23        def forward(self, x, mask=None):
24            x = x + self.mha(self.ln1(x), mask=mask)
25            x = x + self.mlp(self.ln2(x))
26            return x

```

2.2.5 文本的分词器

我们的词汇表是 `ascii` 码，所以 `vocab_size = 256`。

```

1  def tokenizer(text, encode=True, mask=None, max_seq_length=32):
2      if encode:
3          out = chr(2) + text + chr(3) # 添加 SOT token 和 EOT token
4          out = out + "".join([
5              chr(0) for _ in range(max_seq_length-len(out))
6          ]) # 添加Padding
7          out = torch.IntTensor(list(out.encode("utf-8"))) # 对文本进行编码
8          mask = torch.ones(len(out.nonzero()))
9          mask = torch.cat((
10              mask,
11              torch.zeros(max_seq_length - len(mask))
12          )).type(torch.IntTensor)
13      else:
14          # 将input_ids解码为text文本
15          out = [chr(x) for x in text[1:len(mask.nonzero())-1]]
16          out = "".join(out)
17          mask = None
18
19      return out, mask

```

Transformer 无法处理原始文本，因此我们需要做的第一件事是在将输入字符串通过文本编码器之前对其进行分词。

在本教程中，我们将进行一个简单版本的分词，其中我们只使用 **UTF-8** 编码。为什么使用 **UTF-8** 编码进行分词？因为我们只会在示例中使用简单的 `ascii` 码文本。对于更复杂的示例，可能需要使用 **BPE** 分词器。这是因为使用 **UTF-8** 编码时，词表大小 (`vocab_size`) 为 **256**，这意味着对于更复杂的示例，可能会有更长的输入序列，由于上下文长度有限，这在计算注意力时效率低下。

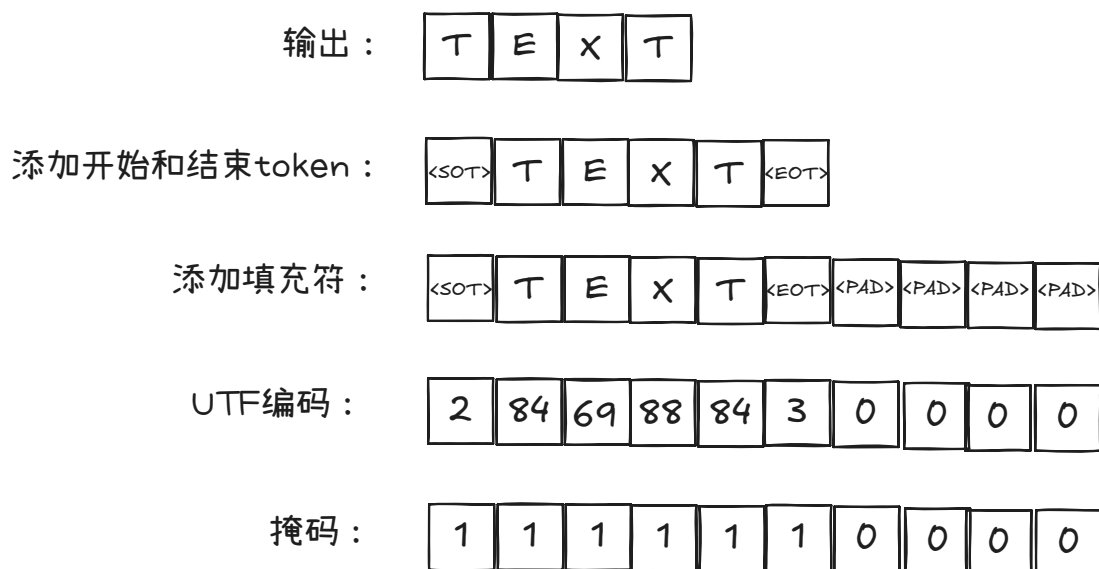


图 2.3 分词过程

分词器的第一步是将文本的开头和文本的结尾 token 添加到输入字符串中。

```
1 text = chr(2) + text + chr(3)
```

[Python](#)

添加文本的开头和文本的结尾 token 后，我们需要将序列的长度填充到最大序列长度。

```
1 text = text + "".join([chr(0) for _ in range(10-len(text))])
```

[Python](#)

我们通过将文本序列编码为 UTF-8 并将输出转换为 IntTensor 来完成分词。

```
1 text = torch.IntTensor(list(text.encode("utf-8")))
```

[Python](#)

对文本进行分词后，我们需要为文本创建掩码。虽然 Transformer 中通常使用的掩码用于确保 token 不与未来的 token 通信，但我们在这里应用的掩码只是使 pad token 被忽略。因此，掩码将只是一个大小等于最大序列长度的张量，其中元素在有填充的情况下为 0，否则为 1。

```
1 mask = torch.ones(len(text.nonzero()))
2 mask = torch.cat((mask, torch.zeros(10-len(mask)))).type(torch.IntTensor)
```

[Python](#)

2.2.6 文本编码器

```
1 class TextEncoder(nn.Module):
2     def __init__(
3         self,
4         vocab_size, # 词汇表大小=256
5         width, # 宽度d_model
6         max_seq_length, # 文本最大长度
7         n_heads,
8         n_layers,
9         emb_dim # 嵌入维度
10    ):
11        super().__init__()
12        self.max_seq_length = max_seq_length
```

[Python](#)

```

13     self.encoder_embedding = nn.Embedding(vocab_size, width)
14     self.positional_embedding = PositionalEmbedding(
15         width,
16         max_seq_length
17     )
18     self.encoder = nn.ModuleList([
19         TransformerEncoder(width, n_heads)
20         for _ in range(n_layers)
21     ])
22     # 可学习投影 (projection)
23     #  $W_{width \times emb\_dim}$ 
24     self.projection = nn.Parameter(torch.randn(width, emb_dim))
25
26     def forward(self, text, mask=None):
27         # 文本嵌入
28         x = self.encoder_embedding(text)
29         # 位置嵌入
30         x = self.positional_embedding(x)
31         # Transformer编码器
32         for encoder_layer in self.encoder:
33             x = encoder_layer(x, mask=mask)
34         # 从EOT的嵌入抽取特征
35         x = x[
36             torch.arange(text.shape[0]), # 批次中数据的索引
37             # 取出掩码mask矩阵的第0行, 加和再减1, 就得到了EOT的索引
38             torch.sub(torch.sum(mask[:, 0], dim=1), 1)
39         ]
40         if self.projection is not None:
41             x = x @ self.projection
42         # 向量x转换为模长为1的向量
43         x = x / torch.norm(x, dim=-1, keepdim=True)
44         return x

```

} (1) 将文本嵌入映射到CLIP嵌入空间

对于文本编码器，我们将使用常规 Transformer 模型。创建文本编码器的第一步是创建大小为 (vocab_size, width) 的嵌入表。此嵌入表包含一个向量表示，其大小等于词汇表中每个 token 的 Transformer 模型的 width。

```
1 self.encoder_embedding = nn.Embedding(vocab_size, width)
```

[Python](#)

在输出 Transformer 的结果之前，我们需要将特征嵌入到联合嵌入空间中。我们将通过获取文本特征的点积以及使用 nn.Parameter 创建的可学习的投影来实现这一点。

```
1 self.projection = nn.Parameter(torch.randn(width, emb_dim))
```

[Python](#)

在 forward 方法中，我们要做的第一件事是通过嵌入表传递文本的 token。

```
1 x = self.encoder_embedding(text)
```

[Python](#)

然后，我们需要将位置编码添加到嵌入表的输出中。

```
1 x = self.positional_embedding(x)
```

[Python](#)

添加位置编码后，我们现在可以将其与掩码一起通过编码器层。

```
1 for encoder_layer in self.encoder:
2     x = encoder_layer(x, mask=mask)
```

Python

编码器层的输出是文本的特征。我们将使用从 EOT 的嵌入中抽取的特征。

```
1 # 从EOT的嵌入抽取特征
2 x = x[torch.arange(
3     text.shape[0]),
4     torch.sub(torch.sum(mask[:,0],dim=1),1)
5 ]
```

Python

最后，我们通过计算特征和投影之间的点积，将文本嵌入映射到 CLIP 嵌入空间中，并通过除以归一化的点积对其进行归一化。

🔥 为什么要做映射？

主要是为了在 CLIP 嵌入空间中，文本嵌入向量的维度和图像嵌入向量的维度一致。向量除以向量的模长，就是模长为 1 的向量。这样文本嵌入向量和图像嵌入向量都变成了模长为 1 的向量。两个向量的点积，就是两个向量的余弦相似度！

```
1 if self.projection is not None:
2     x = x @ self.projection
3 x = x / torch.norm(x, dim=-1, keepdim=True)
4 return x
```

Python

2.2.7 图像编码器

```
1 class ImageEncoder(nn.Module):
2     def __init__(
3         self,
4         width, # 补丁嵌入的维度d_model
5         img_size,
6         patch_size,
7         n_channels,
8         n_layers,
9         n_heads,
10        emb_dim
11    ):
12        super().__init__()
13
14        assert img_size[0] % patch_size[0] == 0 \
15            and img_size[1] % patch_size[1] == 0, \
16            "img_size必须能被patch_size整除"
17        assert width % n_heads == 0, \
18            "width必须能被n_heads整除"
19
20        self.n_patches = (img_size[0] * img_size[1]) \
21            // (patch_size[0] * patch_size[1])
22        self.max_seq_length = self.n_patches + 1
23        self.linear_project = nn.Conv2d(
24            n_channels,
```

Python


```

25         width,
26         kernel_size=patch_size,
27         stride=patch_size
28     )
29     self.cls_token = nn.Parameter(torch.randn(1, 1, width))
30     self.positional_embedding = PositionalEmbedding(
31         width,
32         self.max_seq_length
33     )
34     self.encoder = nn.ModuleList([
35         TransformerEncoder(width, n_heads)
36         for _ in range(n_layers)
37     ])
38
39     #  $W_{width \times emb\_dim}$ 
40     self.projection = nn.Parameter(torch.randn(width, emb_dim))
41
42     def forward(self, x):
43         # 补丁嵌入
44         x = self.linear_project(x)
45         x = x.flatten(2).transpose(1, 2)
46
47         # 位置嵌入
48         x = torch.cat((self.cls_token.expand(x.size()[0], -1, -1), x), dim=1)
49         x = self.positional_embedding(x)
50
51         # Transformer编码器
52         for encoder_layer in self.encoder:
53             x = encoder_layer(x)
54
55         # 获取类别token
56         x = x[:, 0, :]
57
58         if self.projection is not None:
59             x = x @ self.projection
60
61         # 向量x转换为模长为1的向量
62         x = x / torch.norm(x, dim=-1, keepdim=True)
63         return x

```

(1) 将图像嵌入映射到CLIP嵌入空间

2.3 CLIP 模型

```

1  class CLIP(nn.Module):
2      def __init__(
3          self,
4          emb_dim, # 经过可学习投影后的嵌入维度
5          vit_width, # 图像编码器的宽度(d_model)
6          img_size,
7          patch_size,
8          n_channels,

```

Python

```

9      vit_layers,
10     vit_heads,
11     vocab_size,
12     text_width, # 文本编码器的宽度 (d_model)
13     max_seq_length,
14     text_heads,
15     text_layers
16 ):
17     super().__init__()
18     self.image_encoder = ImageEncoder(
19         vit_width,
20         img_size,
21         patch_size,
22         n_channels,
23         vit_layers,
24         vit_heads,
25         emb_dim
26     )
27     self.text_encoder = TextEncoder(
28         vocab_size,
29         text_width,
30         max_seq_length,
31         text_heads,
32         text_layers,
33         emb_dim
34     )
35     # 可学习温度
36     self.temperature = nn.Parameter(torch.ones([1]) * np.log(1 / 0.07))
37     self.device = torch.device("cuda")
38
39
40     def forward(self, image, text, mask=None):
41         #  $I_e$  是图像嵌入, 形状 [B, D=emb_dim]
42         I_e = self.image_encoder(image)
43         #  $T_e$  是文本嵌入, 形状 [B, D=emb_dim]
44         T_e = self.text_encoder(text, mask=mask)
45
46         # 缩放逐点余弦相似度 [n, n]
47         # 形状  $I_e \cdot T_e : [B, D] @ [D, B] \rightarrow [B, B]$ 
48         logits = (I_e @ T_e.transpose(-2, -1)) * torch.exp(self.temperature)
49
50         # 对称损失函数: labels 形状为 [B], 值为 [0, 1, 2, ..., B-1]
51         labels = torch.arange(logits.shape[0]).to(self.device)
52         # 从文本  $\rightarrow$  图像方向, 以文本嵌入  $T_3$  为例子,
53         # 交叉熵损失的目标是让  $T_3 \cdot I_3$  越大越好
54         loss_i = nn.functional.cross_entropy(
55             logits.transpose(-2, -1),
56             labels
57         )

```

```

58     # 从图像 → 文本方向，以图像嵌入  $I_3$  为例子，
59     # 交叉熵损失的目标是让  $I_3 \cdot T_3$  越大越好
60     loss_t = nn.functional.cross_entropy(
61         logits,
62         labels
63     )
64     # 两个方向的损失求平均值
65     loss = (loss_i + loss_t) / 2
66
67     return loss

```

当给定一批图像和标题时，CLIP 应该告诉我们哪些标题与哪些图像搭配。它通过一起训练文本编码器和图像编码器来最大化应该在一起的对的成对余弦相似度分数，并最小化不应该在一起的对来做到这一点。

为此，我们首先需要从图像和文本编码器中获取特征的嵌入。

```

1 def forward(self, image, text, mask=None):
2     I_e = self.image_encoder(image)
3     T_e = self.text_encoder(text, mask=mask)

```

Python

使用嵌入的特征，我们可以通过使用嵌入的图像特征和嵌入文本特征的转置版本之间的点积来计算缩放的成对余弦相似度。余弦相似度应沿图中正确的图像和文本配对在一起的对角线最大化。

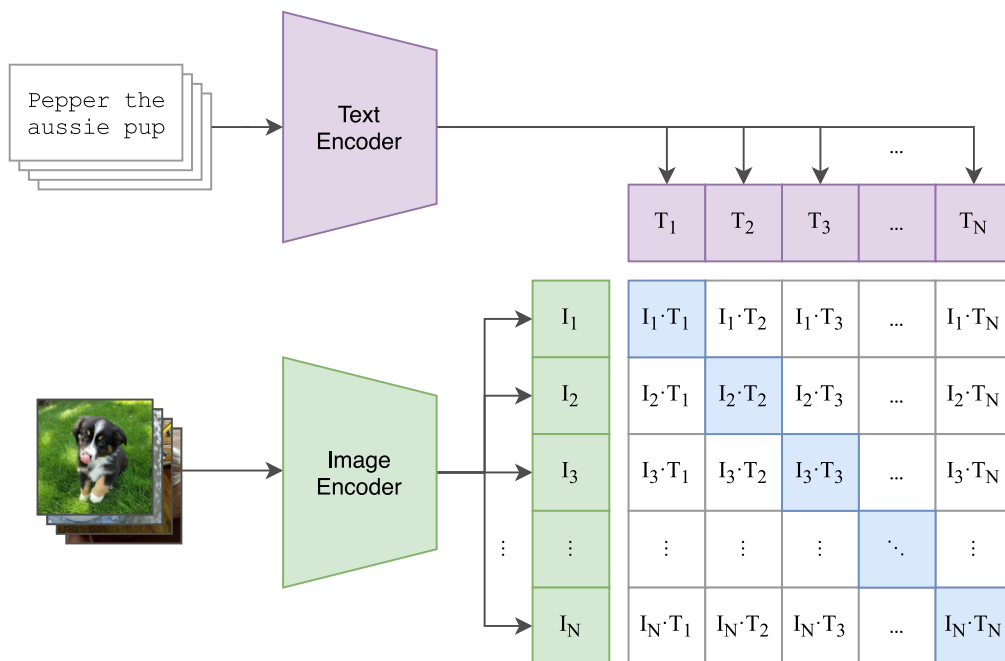


图 2.4 余弦相似度的计算

```

1 logits = (I_e @ T_e.transpose(-2, -1)) * torch.exp(self.temperature)

```

Python

这在 I_e 和 T_e 分别包含 N 个批次时工作，从而产生上图所示的矩阵。为了最大化相关图像之间的余弦相似度，CLIP 使用对称/对比损失。我们可以通过首先创建与批次中的每条数据相对应的标签来计算此损失。

```

1 # 对称损失函数
2 labels = torch.arange(logits.shape[0]).to(self.device)

```

Python

然后，我们计算沿 **logit** 行的交叉熵损失，以获得图像的损失。

```
1 loss_i = nn.functional.cross_entropy(logits.transpose(-2,-1), labels)
```

 Python

文本的损失是通过计算沿列的交叉熵损失来计算的。

```
1 loss_t = nn.functional.cross_entropy(logits, labels)
```

 Python

我们通过计算图像损失和文本损失之间的平均值来获得最终损失。

```
1 loss = (loss_i + loss_t) / 2
2 return loss
```

 Python

2.3.1 数据

```
1 class MNIST(Dataset):
2     def __init__(self, train=True):
3         self.dataset = load_dataset("../datasets/clip-mnist/")
4         self.transform = T.ToTensor()
5         if train:
6             self.split = "train"
7         else:
8             self.split = "test"
9
10        self.captions = {
11            0: "An image of Zero",
12            1: "An image of One",
13            2: "An image of Two",
14            3: "An image of Three",
15            4: "An image of Four",
16            5: "An image of Five",
17            6: "An image of Six",
18            7: "An image of Seven",
19            8: "An image of Eight",
20            9: "An image of Nine"
21        }
22
23    def __len__(self):
24        return self.dataset.num_rows[self.split]
25
26    def __getitem__(self, i):
27        # 取出第i张图片
28        img = self.dataset[self.split][i]["image"]
29        # 转换成张量
30        img = self.transform(img)
31        # 图片对应的文本，以及掩码
32        cap, mask = tokenizer(
33            self.captions[self.dataset[self.split][i]["label"]]
34        )
35        # 为什么要repeat?
36        mask = mask.repeat(len(mask), 1)
37        return {"image": img, "caption": cap, "mask": mask}
```

 Python

在本教程中，我们将使用 MNIST 数据集。我们选择这个数据集是因为它相当小并且保持训练时间合理。

```
1 self.dataset = load_dataset("clip-mnist")
```

[Python](#)

对于数据集中的每个条目，我们将需要 3 样东西：图像、文本和文本掩码。

对于图像，我们唯一需要进行的更改是将图像转换为张量。

```
1 img = self.dataset[self.split][i]["image"]
2 img = self.transform(img)
```

[Python](#)

对于标题，我们需要将其传递给我们创建的分词器，以获取 token 表示以及 token 的掩码。

```
1 cap, mask = tokenizer(self.captions[self.dataset[self.split][i]["label"]])
```

[Python](#)

我们从分词器获得的掩码是大小为 `max_seq_length` 的 1 维张量。在文本编码器中，掩码将应用于形状为 `(max_seq_length, max_seq_length)` 的注意力分数。因此，我们需要扩展掩码，以便将其应用于注意力分数的每一行。

原始掩码：

1	1	1	0	0
---	---	---	---	---

新掩码：

1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0

图 2.5 复制掩码

```
1 mask = mask.repeat(len(mask), 1)
```

[Python](#)

图像、标题和掩码作为字典保存在数据集中。

```
1 return {"image": img, "caption": cap, "mask": mask}
```

[Python](#)

提示

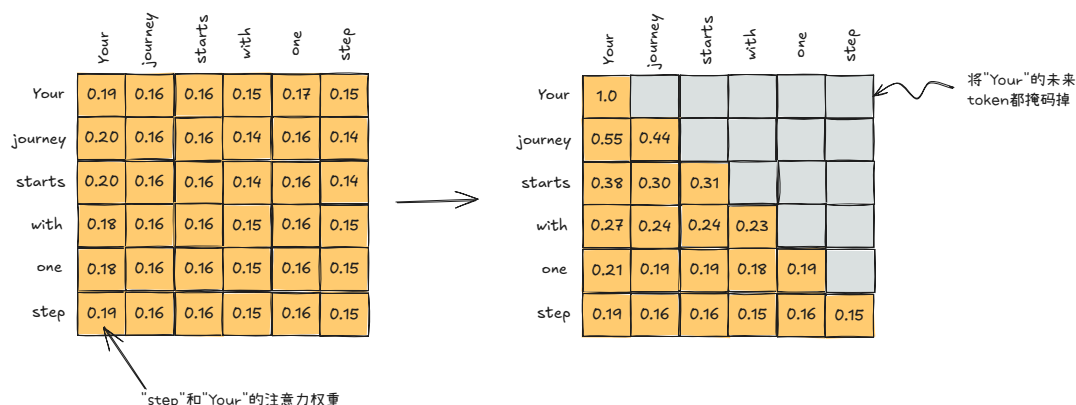


图 2.6 因果注意力分数的掩码

创建 [context_length, context_length] 的上三角矩阵。

```
1 self.register_buffer(
2     'mask',
3     torch.triu(torch.ones(
4         context_length,
5         context_length
6     ), diagonal = 1))
```

Python

将注意力分数的上三角置为 -torch.inf, 下三角保留。得到注意力分数的下三角矩阵。

```
1 attention_score.masked_fill_(
2     self.mask[:seq_len, :seq_len].bool(),
3     -torch.inf
4 )
```

Python

2.3.2 训练参数

```
1 emb_dim = 32 # 文本编码器和图像编码器输出的张量的维度
2 vit_width = 9 # 图像编码器的嵌入的宽度
3 img_size = (28,28)
4 patch_size = (14,14)
5 n_channels = 1
6 vit_layers = 3 # 图像编码器中编码器的层的数量
7 vit_heads = 3 # 图像编码器中注意力头的数量
8 vocab_size = 256 # 词汇表大小
9 text_width = 32 # 文本编码器的嵌入的宽度
10 max_seq_length = 32 # 最大序列长度
11 text_heads = 8 # 文本编码器中注意力头的数量
12 text_layers = 4 # 文本编码器中编码器的层数
13 lr = 1e-3 # 学习率
14 epochs = 10
15 batch_size = 128
```

Python

2.3.3 加载数据集

```
1 train_set = MNIST(train = True)
2 test_set = MNIST(train = False)
3
4 train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
5 test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

 Python

2.3.4 训练模型

```
1 device = torch.device("cuda")
2
3 model = CLIP(
4     emb_dim,
5     vit_width,
6     img_size,
7     patch_size,
8     n_channels,
9     vit_layers,
10    vit_heads,
11    vocab_size,
12    text_width,
13    max_seq_length,
14    text_heads,
15    text_layers
16 ).to(device)
17
18 optimizer = optim.Adam(model.parameters(), lr=lr)
19
20 best_loss = np.inf
21 for epoch in range(epochs):
22     for i, data in enumerate(train_loader, 0):
23         img = data["image"].to(device)
24         cap = data["caption"].to(device)
25         mask = data["mask"].to(device)
26         loss = model(img, cap, mask)
27         optimizer.zero_grad()
28         loss.backward()
29         optimizer.step()
30
31     print(f"Epoch [{epoch+1}/{epochs}], Batch Loss: {loss.item():.3f}")
32
33     # 保存模型
34     if loss.item() <= best_loss:
35         best_loss = loss.item()
36         torch.save(model.state_dict(), "./clip.pt")
37     print("模型已经保存.")
```

 Python

2.3.5 评估模型

```

1  # 加载最好的模型
2  model = CLIP(
3      emb_dim,
4      vit_width,
5      img_size,
6      patch_size,
7      n_channels,
8      vit_layers,
9      vit_heads,
10     vocab_size,
11     text_width,
12     max_seq_length,
13     text_heads,
14     text_layers
15 ).to(device)
16 model.load_state_dict(torch.load("./clip.pt", map_location=device))
17
18 # 获取数据集的标签和图片进行对比
19 text = torch.stack(
20     [tokenizer(x)[0] for x in test_set.captions.values()]
21 ).to(device)
22 mask = torch.stack(
23     [tokenizer(x)[1] for x in test_set.captions.values()]
24 )
25 mask = mask.repeat(
26     1,
27     len(mask[0])
28 ).reshape(
29     len(mask),
30     len(mask[0])
31 ), len(mask[0])).to(device)
32
33 correct, total = 0, 0
34 with torch.no_grad():
35     for data in test_loader:
36         # 图像
37         images = data["image"].to(device)
38         # 文本
39         labels = data["caption"].to(device)
40         # 使用clip模型中的图像编码器对图像抽取特征
41         image_features = model.image_encoder(images)
42         # 使用clip模型中的文本编码器对文本抽取特征
43         text_features = model.text_encoder(text, mask=mask)
44         # 转换为模长为1的向量
45         image_features /= image_features.norm(dim=-1, keepdim=True)
46         text_features /= text_features.norm(dim=-1, keepdim=True)
47         #  $I_e @ T_e^T: I_e \cdot T_e$ 
48         similarity = (

```



```

49         100.0 * image_features @ text_features.T
50     ).softmax(dim=-1)
51     _, indices = torch.max(similarity, 1)
52     # 预测结果
53     pred = torch.stack([
54         tokenizer(test_set.captions[int(i))][0]
55         for i in indices
56     ]).to(device)
57     # 预测正确的样本数量
58     correct += int(sum(torch.sum((pred==labels),dim=1)//len(pred[0])))
59     total += len(labels)
60
61 print(f'\n预测准确率: {100 * correct // total} %')

```

我们通过获取模型训练的标题并将其与实际标题进行比较来测试模型。在训练时，我们使用了相同的标题模板（"A image of a(n) {class}"），因此这个测试阶段与任何其他图像分类器几乎相同。我们实现了大约 85% 的模型准确率。

2.3.6 零样本分类

```

1  # 加载模型
2  model = CLIP(
3      emb_dim,
4      vit_width,
5      img_size,
6      patch_size,
7      n_channels,
8      vit_layers,
9      vit_heads,
10     vocab_size,
11     text_width,
12     max_seq_length,
13     text_heads,
14     text_layers
15 ).to(device)
16 model.load_state_dict(torch.load("./clip.pt", map_location=device))
17
18 # 标题
19 class_names = [
20     "a photo of 0",
21     "an image of one",
22     "2",
23     "3",
24     "4",
25     "5",
26     "6",
27     "7",
28     "8",
29     "9",
30     "Trump",

```

 Python

```

31     "Musk"
32 ]
33
34 text = torch.stack(
35     [tokenizer(x)[0] for x in class_names]
36 ).to(device)
37 mask = torch.stack(
38     [tokenizer(x)[1] for x in class_names]
39 )
40 mask = mask.repeat(
41     1,
42     len(mask[0])
43 ).reshape(len(mask), len(mask[0]), len(mask[0])).to(device)
44
45 idx = 1000
46 # 去测试数据集中的第1000张图片
47 img = test_set[idx]["image"][None,:]
48 plt.imshow(img[0].permute(1, 2, 0), cmap="gray")
49 # 将图片的标题文本展示, 例如"An Image Of Nine"
50 plt.title(tokenizer(
51     test_set[idx]["caption"],
52     encode=False,
53     mask=test_set[idx]["mask"][0]
54 )[0])
55 plt.show()
56 img = img.to(device)
57 with torch.no_grad():
58     # 抽取图片的特征
59     image_features = model.image_encoder(img)
60     # 抽取文本的特征
61     text_features = model.text_encoder(text, mask=mask)
62
63 image_features /= image_features.norm(dim=-1, keepdim=True)
64 text_features /= text_features.norm(dim=-1, keepdim=True)
65 # 计算第1000张图片和所有文本的相似度
66 similarity = (
67     100.0 * image_features @ text_features.T
68 ).softmax(dim=-1)
69 # 返回所有文本中和图片特征最相似的5个文本
70 values, indices = similarity[0].topk(5)
71
72 # 打印结果
73 print("\n预测结果:\n")
74 for value, index in zip(values, indices):
75     print(f"{class_names[int(index)]:>16s}: {100 * value.item():.2f}%")

```

对于 **zero-shot** 分类, 我们将图像与类别的名称进行比较。我们输入标签以与图像进行比较, 它将返回前 5 个预测以及预测的可能性。这不是 CLIP 执行 **zero-shot** 分类的最佳示例。使用 MNIST 数据集使模型易于训练, 但标题不是很丰富。要真正理解 CLIP 的 **zero-shot** 功能, 包含多个名称的训练集会更适合。真正的 **zero-shot** 检测将允许检测以前未见过的排列。

 提示

具体步骤如下：

1. 将数据库中的所有图片使用 `clip` 的图像编码器进行抽取特征
2. 将抽取的图片特征存入向量数据库
3. 将输入的搜索文本通过 `clip` 的文本编码器抽取文本特征
4. 使用余弦相似度找出向量数据库中和文本特征最接近的几张图片

3. ClipCap (图生文)

🔥 Image Captioning Model

图片标题模型(image captioning model)是一种将图片作为输入并生成图片描述模型。

下面是一个简单的示例，展示了一个图片标题模型：

Image Captioning Model

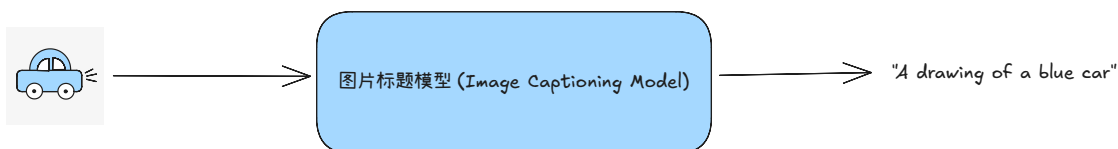


图 3.1 图片标题模型的简单示例

3.1 ClipCap 的工作原理

ClipCap 是一种结合了 CLIP 和 GPT-2 的图片标题架构。

CLIP 是我们将来用来创建输入图像嵌入的模型。

GPT-2 是一种基于解码器的模型，用于生成文本。

ClipCap 的基本工作原理如下：

输入图像首先通过 CLIP 模型转换为嵌入，目的是利用这种嵌入(捕捉图像意义)来引导 GPT-2 生成文本。

但有一个问题：CLIP 和 GPT-2 的嵌入空间不同。所以我们不能直接把这个嵌入输入到 GPT-2 中。

为了解决这个问题，我们使用一个映射网络将 CLIP 嵌入映射到 GPT-2 的嵌入空间。

这些映射的图像嵌入称为前缀(prefixes)，因为它们是 GPT-2 生成图片说明所需的上下文。

我们将使用这些嵌入作为“前缀”(prefix)来使得 GPT2 生成标题

但问题是 CLIP 和 GPT2 有着不同的嵌入空间，所以必须将 CLIP 嵌入映射到 GPT2 的嵌入空间，才能使用这些嵌入

将图片的嵌入从 CLIP 的嵌入空间映射到 GPT2 的嵌入空间

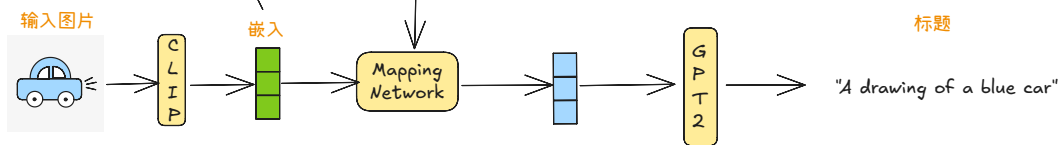


图 3.2 将图片的 CLIP 嵌入映射到 GPT2 的嵌入空间

3.2 关于训练

CLIP 生成的图像嵌入开箱即用已经足够好——所以我们不训练 CLIP 模型。

根据 GPT-2 是否经过微调，ClipCap 有两种变体：

- 如果我们对 GPT-2 进行微调，那么我们就用 MLP 作为映射网络。GPT-2 和 MLP 都经过训练。
- 如果我们不对 GPT-2 进行微调，那么我们就用 Transformer 架构（例如 Bert）作为映射网络，只有变换器本身被训练。

就我而言，我选择对 GPT-2 模型进行微调，并使用 MLP 作为映射网络。

3.3 推理

在我们这里，这意味着为未见的图片生成说明文字。

让我们用一辆蓝色汽车的图片作为推理过程的例子：

1. 输入图像被转换为 CLIP 嵌入。
2. 该嵌入通过映射网络传递，生成前缀嵌入。
3. 在时间步 $t=1$ 时，我们将这些前缀嵌入传递到 GPT2。模型预测下一个 token——假设是“a”。我们将此附加到序列中。
4. 在 $t=2$ 时，我们将更新后的输入序列（前缀+“a”）传递给 GPT2。它预测下一个 token——这次可能是“蓝色”。
5. 这一过程持续进行，模型一次预测一个 token，直到它：
 - 生成 <EOS>（序列结束）token，或
 - 生成的标题长度达到最大。

如下图所示

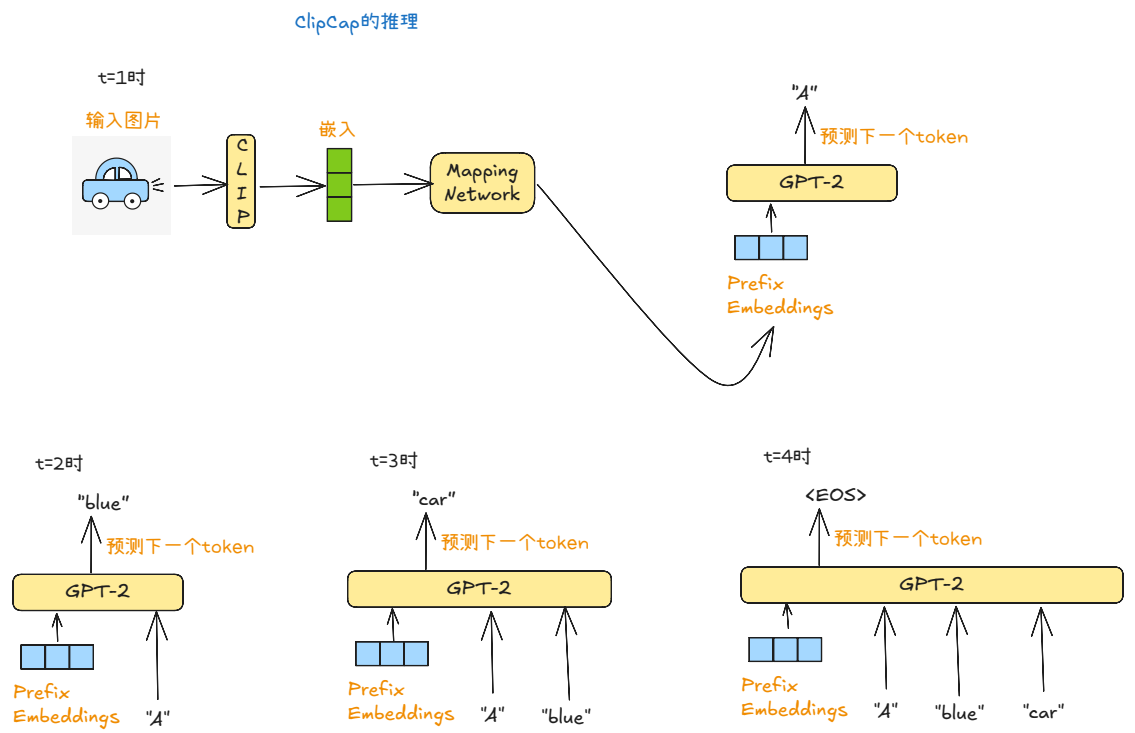


图 3.3 ClipCap 的推理过程

3.4 ClipCap 的代码实现

4. 扩散模型

扩散模型的兴起可以看作是近年来 AI 生成艺术作品领域取得突破的主要因素。

在图像创作方面,扩散模型已成为内容生成领域的前沿技术。尽管该模型于 2015 年首次推出,但已取得显著进展,并已成为 DALL·E 和 Midjourney 等知名模型的核心机制。



DDPM 算法

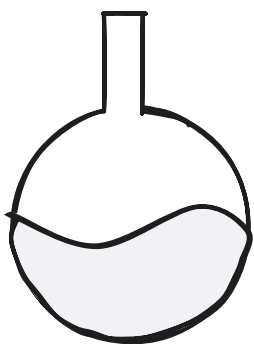
Denoising Diffusion Probabilistic Models, 去噪扩散概率模型

4.1 通俗讲解

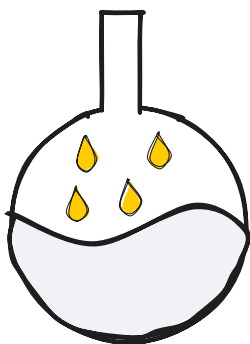
4.1.1 扩散

4.1.1.1 物理学中的类比

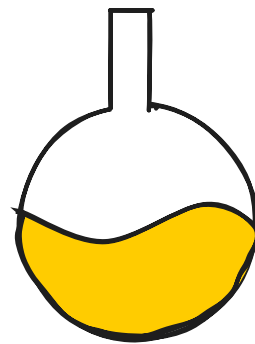
想象一下一杯透明的水。如果我们加入少量其他颜色的液体,比如黄色的液体,会发生什么?黄色液体会逐渐均匀地扩散到整个玻璃杯中,最终的混合物会呈现出略带透明的黄色。



透明的水



添加了一点黄色液体



水不再透明

图 4.1 水滴的扩散过程

上面的过程被称为 **正向扩散**：我们通过添加少量其他液体来改变环境状态。然而，进行 **反向扩散**——将混合物恢复到其原始状态——是否同样容易？事实证明并非如此。即使在最好的情况下，实现这一点也需要高度复杂的机制。

4.1.1.2 将类比应用于机器学习

扩散也可以应用于图像。想象一下一张高质量的狗狗照片。我们可以通过逐渐添加随机噪声来轻松地变换这幅图像。结果，像素值会发生变化，使图像中的狗狗变得不那么明显，甚至无法辨认。这个变换过程称为 **正向扩散**。

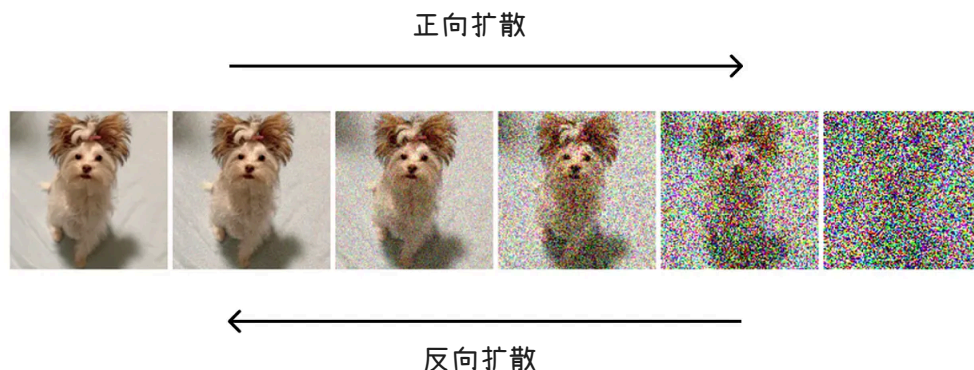


图 4.2 高清图像的扩散过程

我们也可以考虑反向操作：给定一张噪声图像，目标是重建原始图像。这项任务更具挑战性，因为与大量可能的噪声变化相比，可高度识别的图像状态要少得多。用前面提到的物理类比，这个过程称为 **反向扩散**。

在本文中，我将通过示意图来解释它的工作原理。

4.1.2 扩散模型的架构

为了更好地理解扩散模型的结构，让我们分别检查两个扩散过程。

4.1.2.1 正向扩散

如前所述，前向扩散涉及逐步向图像添加噪声。然而，在实践中，这个过程要更加微妙一些。

最常见的方法是从均值为 0 的 **高斯分布** 中为图片中的每个像素采样一个随机值。然后将这个采样值（可以是正值也可以是负值）添加到像素的原始值中。对所有像素重复此操作会得到原始图像的噪声版本。

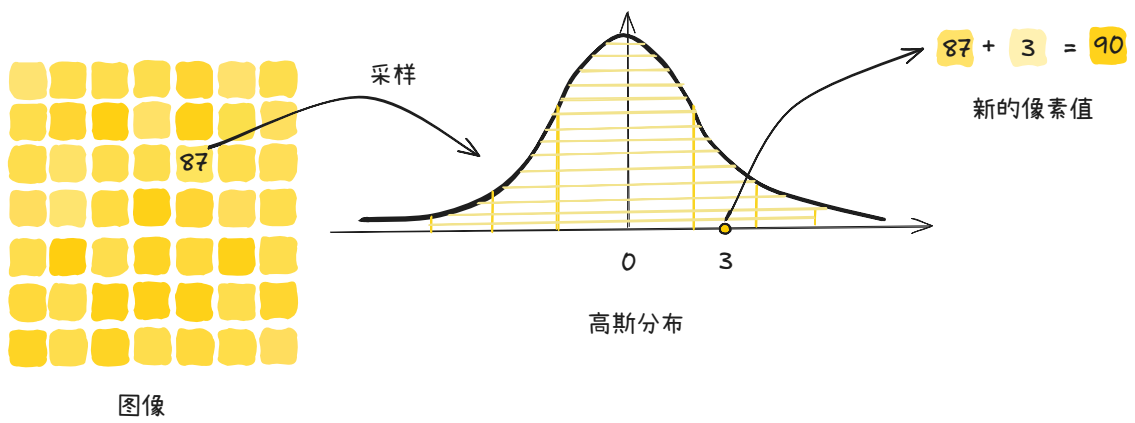


图 4.3 从高斯分布中采样一个随机值

 通知

所选的高斯分布通常方差较小，这意味着采样值通常较小。因此，每一步只会对图像产生微小的变化。

图中有 49 个像素点，那么需要从一个相同的高斯分布中采样 49 次噪声，然后将 49 个噪声分别加到 49 个像素点上。

正向扩散是一个迭代过程，其中噪声被多次应用于图像。随着每次迭代，生成的图像与原始图像的差异越来越大。经过数百次迭代（这在实际扩散模型中很常见）后，图像最终变得无法从纯噪声中识别出来。

4.1.2.2 反向扩散

现在你可能会问：执行所有这些正向扩散变换的目的是什么？答案是，每次迭代生成的图像都用于训练神经网络。

具体来说，假设我们在正向扩散过程中应用了 100 次连续噪声变换。然后，我们可以在每一步获取图像，并训练神经网络重建上一步的图像。预测图像与实际图像之间的差异使用损失函数计算——例如均方误差（MSE），它衡量两幅图像之间的平均像素差异。

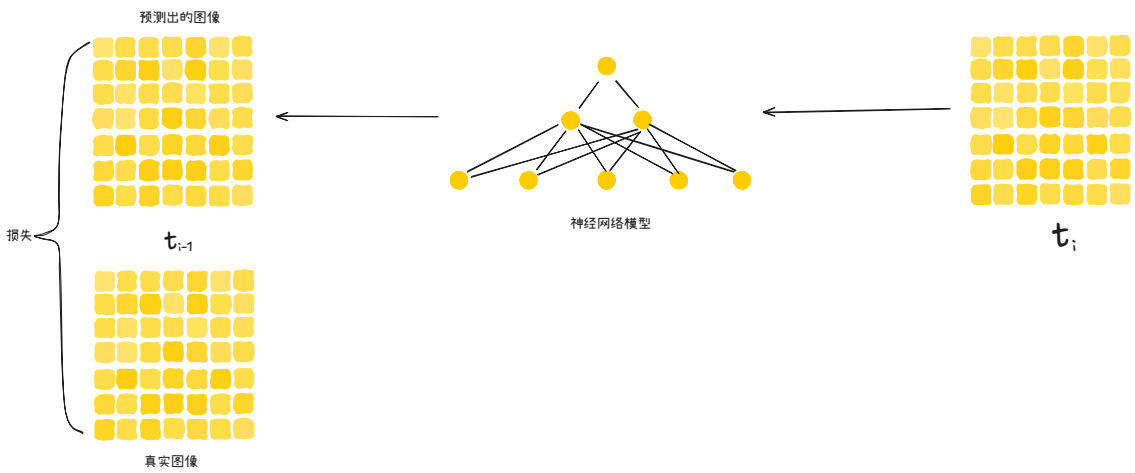


图 4.4 预测图像与实际图像之间的差异使用损失函数计算

🔔 通知

该模型的目标是检测添加的噪声并重建先前的图像。然后将预测图像与实际图像进行比较以计算损失。

这个例子展示了扩散模型重建原始图像的过程。同时，扩散模型可以被训练来预测添加到图像中的噪声。在这种情况下，要重构原始图像，只需要从前一次迭代的图像中减去预测的噪声就足够了。

虽然这两个任务看起来可能相似，但预测添加的噪声比图像重构要简单。

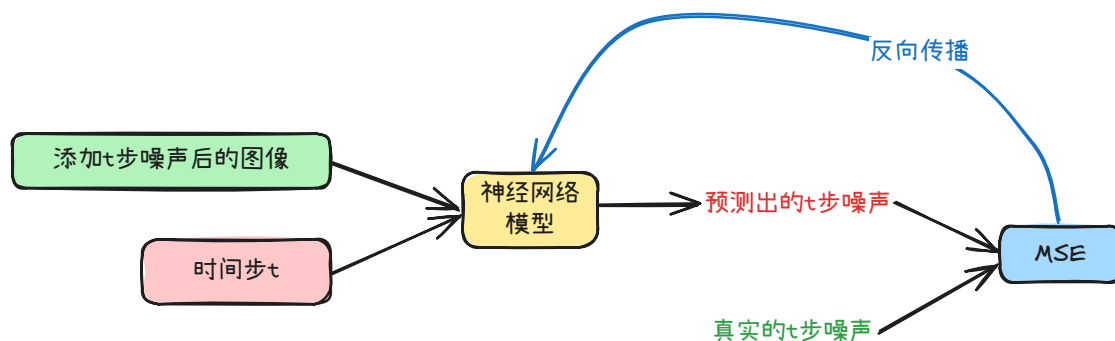


图 4.5 将噪声作为预测目标

4.1.3 模型设计

在对扩散技术有了基本的了解之后，有必要探索一些更高级的概念，以更好地理解扩散模型设计。

4.1.3.1 迭代次数

迭代次数是扩散模型中的关键参数之一：

🔔 通知

一方面，使用更多迭代次数意味着相邻步骤中的图像对差异会更小，从而使模型的学习任务更容易。另一方面，更高的迭代次数会增加计算成本。

虽然较少的迭代次数可以加快训练速度，但模型可能无法学习步骤之间的平滑过渡，从而导致性能不佳。

通常，迭代次数选择在 50 到 1000 之间。

4.1.3.2 神经网络架构

最常见的是，U-Net 架构被用作扩散模型的主干。以下是一些原因：

- U-Net 保留了输入和输出图像的尺寸，确保在整个逆扩散过程中图像大小保持一致。
- 其瓶颈架构能够在将整幅图像压缩到潜在空间后将其重建。同时，通过残差连接保留关键图像特征。
- U-Net 最初设计用于生物医学图像分割，其中像素级精度至关重要，它的优势可以很好地转化为需要精确预测单个像素值的扩散任务。

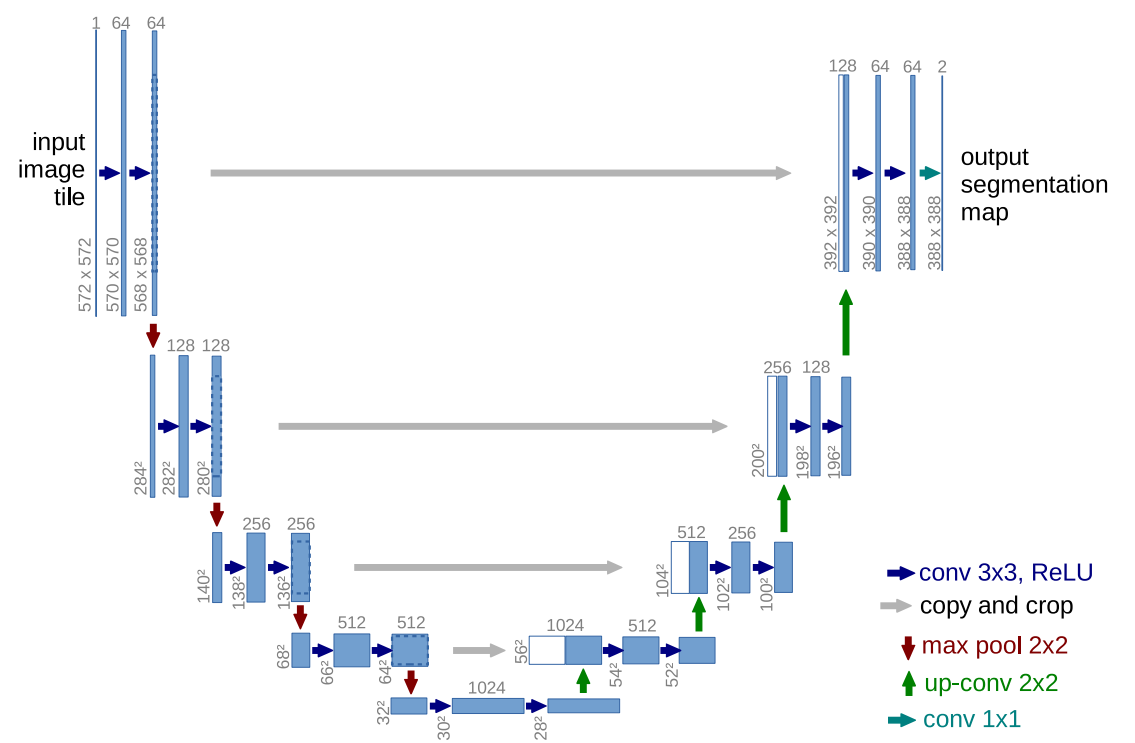


图 4.6 U-net 架构 (以最低分辨率为 32×32 像素为例)。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 $x-y$ 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。

4.1.3.3 共享网络

乍一看，似乎有必要为扩散过程的每次迭代训练一个单独的神经网络。虽然这种方法可行，并且可以产生高质量的推理结果，但从计算角度来看，它效率极低。例如，如果扩散过程包含 1000 个时间步，我们就需要训练 1000 个 U-Net 模型——这是一项极其耗时且资源密集的任务。

然而，我们可以观察到，不同迭代中的任务配置本质上是相同的：在每种情况下，我们都需要重建一个尺寸相同、且经过相似幅度噪声改变的图像。这一重要洞见促成了在所有迭代中使用单个共享神经网络的想法。

实际上，这意味着我们使用一个具有共享权重的 U-Net 模型，该模型基于来自不同扩散步骤的图像对进行训练。在推理过程中，含噪图像会多次通过同一个经过训练的 U-Net 模型，逐步进行优化，直到生成高质量的图像。

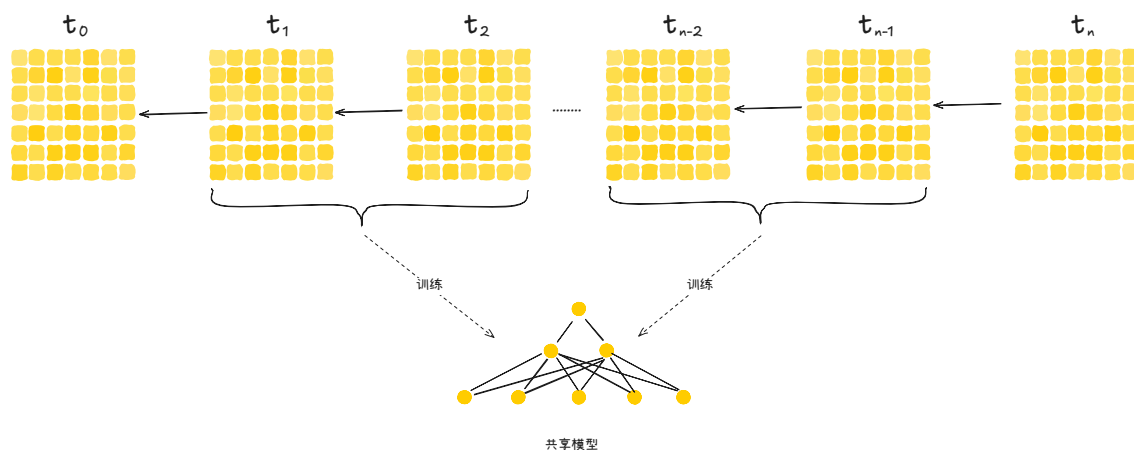


图 4.7 共享模型

虽然由于仅使用单一模型，生成质量可能会略有下降，但训练速度的提升却非常显著。

4.2 扩散模型理论简介

4.2.1 概述

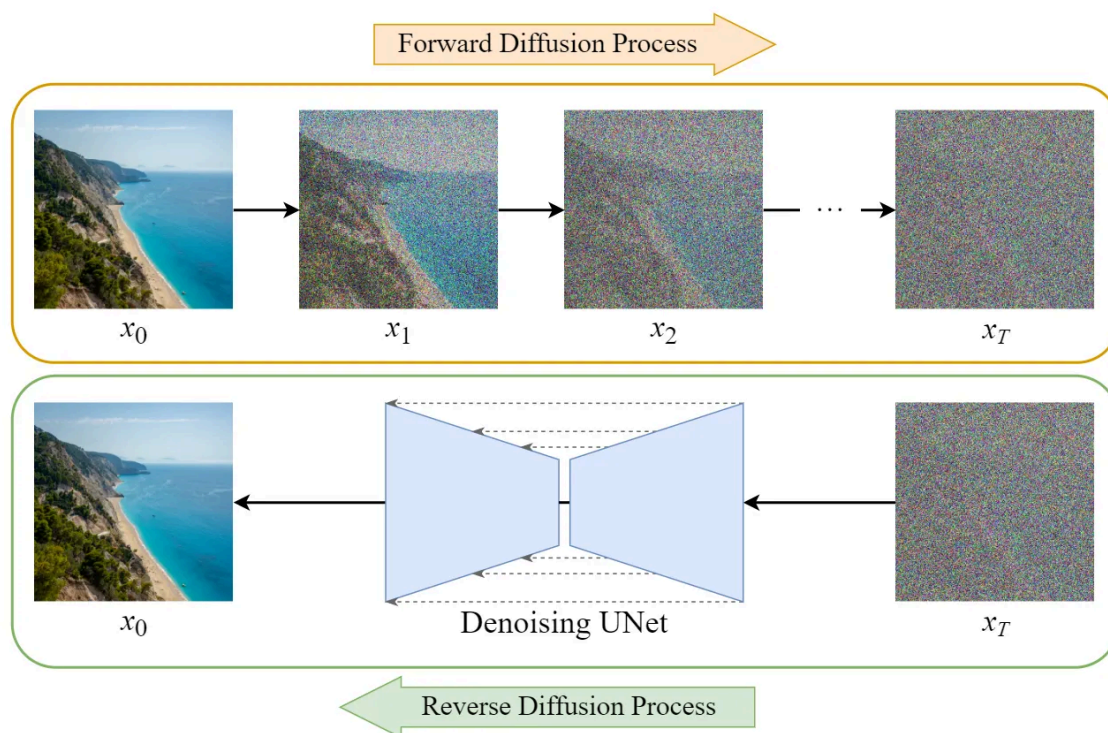


图 4.8 正向扩散和逆向扩散

扩散模型的训练可以分为两部分：

1. 正向扩散过程 → 给图像添加噪声。
2. 反向扩散过程 → 从图像中去除噪声。

4.2.2 正向扩散过程

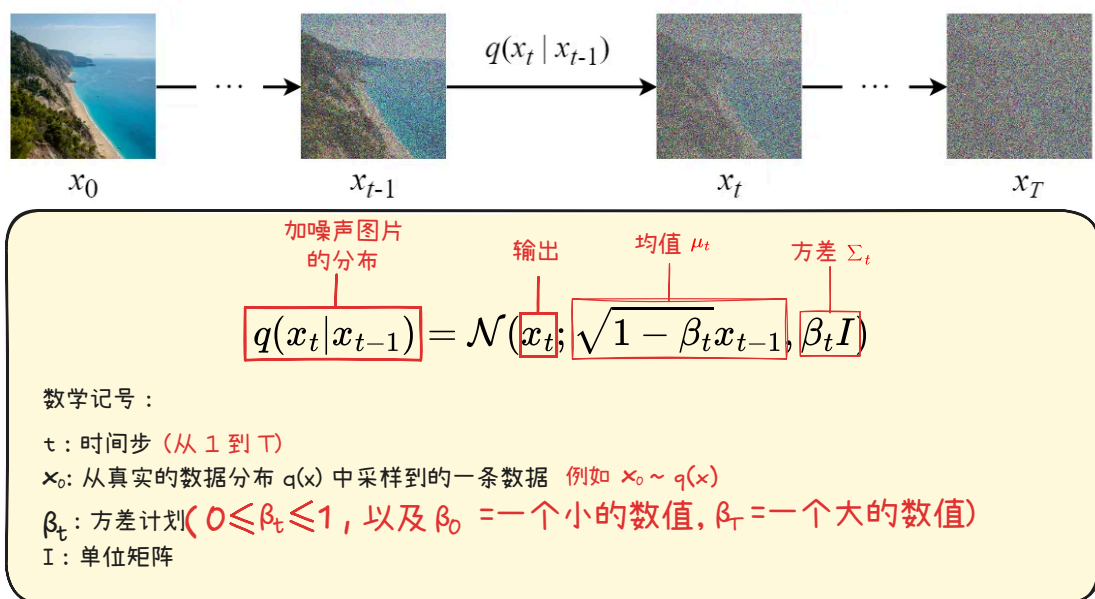


图 4.9 正向扩散公式图解

🔔 方差计划

假设 $\beta_{\text{start}} = 0.0002, \beta_{\text{end}} = 0.04$ ，则第 1 步添加的高斯噪声的方差是 0.0002，第 2 步添加的高斯噪声的方差是 $0.0002 + \frac{0.04 - 0.0002}{1000}$ 。

前向扩散过程逐步将高斯噪声添加到输入图像 x_0 中，总共会有 T 步。该过程将产生一系列带噪声的图像样本 $x_1, x_2, \dots, x_T$ 。

当 $T \rightarrow \infty$ 时，最终结果将变成完全噪声图像，就像从各向同性的高斯分布中采样出来的噪声一样。

首先，如果 $z \sim \mathcal{N}(\mu, \sigma^2)$ 的话，那么正态分布可以写成如下公式：

$$z = \mu + \sigma \varepsilon \quad \text{其中 } \varepsilon \sim \mathcal{N}(0, 1) \quad (4.1)$$

利用这个技巧，我们可以将采样图像 x_t 表示如下：

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_{t-1} \quad (4.2)$$

⚡ 一步一步的添加噪声太麻烦了！

根据上面的公式，如果想要从原始图片 x_0 得到添加了 500 步噪声的图片 x_{500} 需要迭代 500 次！

但我们不需要设计一种算法来迭代地向图像中添加噪声，而是可以使用闭式公式（解析解）在特定的时间步长 t 直接对噪声图像进行采样。

给定原始图片 x_0 和时间步 t 可以直接得到添加了 t 步噪声的图像 x_t 。公式如下：

🔥 给定原始图片 x_0 和时间步 t 直接采样出 x_t 的公式

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad (4.3)$$

其中:

- $\bar{\alpha}_t = \alpha_t \alpha_{t-1} \dots \alpha_1$
- $\alpha_t = 1 - \beta_t$
- $\varepsilon \sim \mathcal{N}(0, 1)$

现在我们可以使用此公式在任何时间步直接对 x_t 进行采样，这使得前向扩散过程更快。

4.2.3 反向扩散过程

反向扩散过程是从一张完全的高斯噪声图片中，逐步去除噪声，来生成一张图片。这个逆向过程很难，相当于一滴墨水在水里面扩散开来（正向扩散），想要将变黑的水逆向为原来的墨水刚滴入水中的状态。这几乎不可能做到，所以我们用神经网络来解决这个问题。下面是训练神经网络的原理。

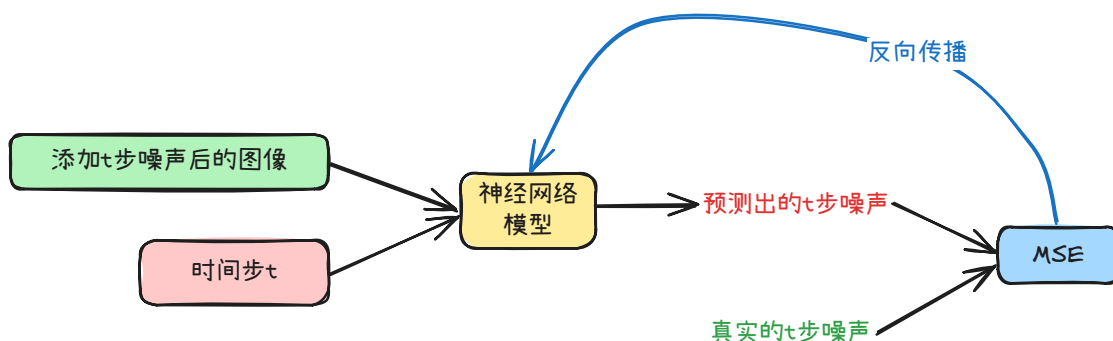


图 4.10 以噪声为预测目标

因此最终的训练目标如下：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad (4.4)$$

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$$

4.2.4 U-Net 模型

4.2.4.1 数据集

在每个 Epoch:

1. 将为每个训练样本（图像）选择一个随机时间步长 t 。
2. 对每幅图像添加高斯噪声（对应于 t ）。
3. 将时间步转换为嵌入（向量）。

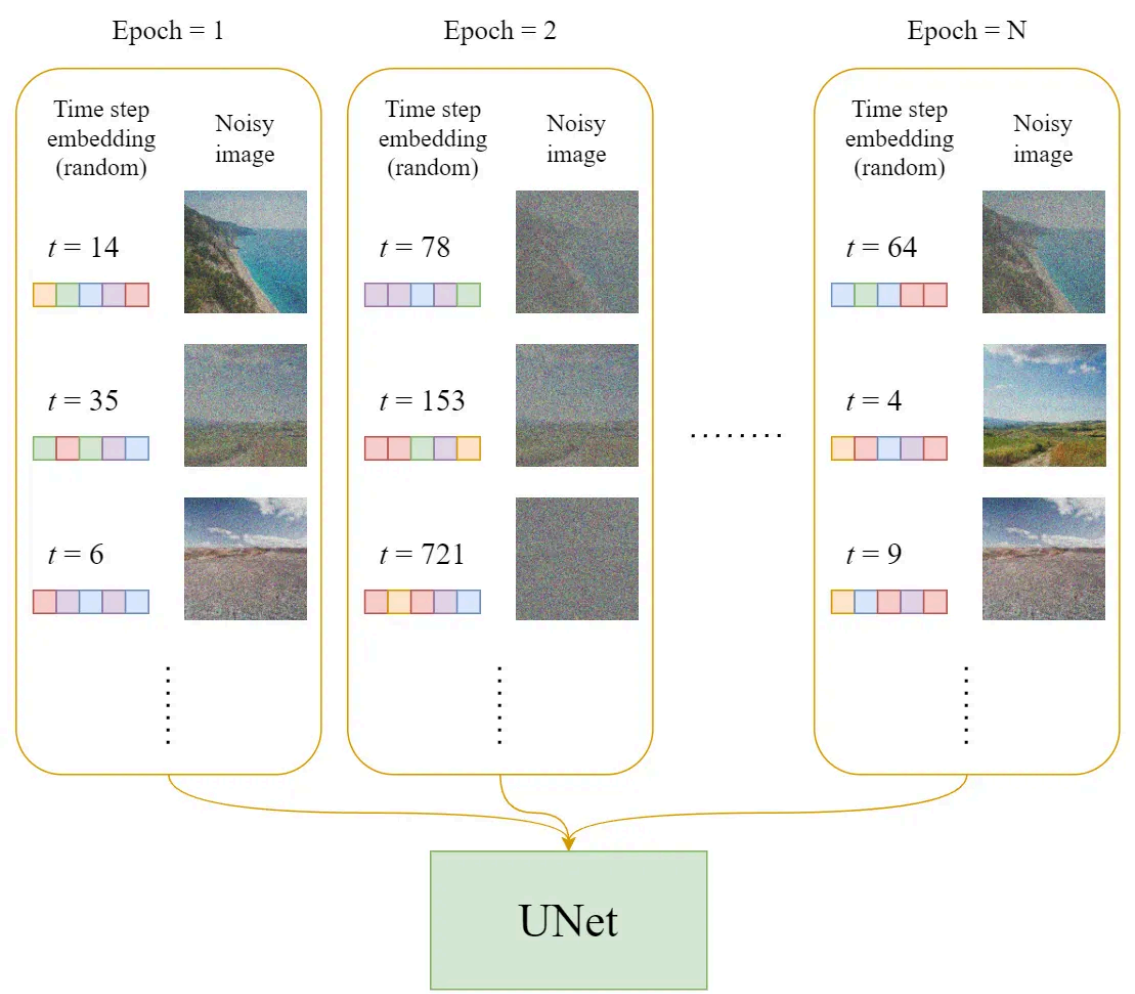


图 4.11 训练 unet

4.2.4.2 训练

1 Repeat

2 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$

3 $t \sim \text{Uniform}(\{1, \dots, T\})$

4 $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$

5 使用梯度下降法，梯度为

6 $\nabla_{\theta} \|\varepsilon - \varepsilon_{\theta}(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\varepsilon, t)\|^2 \quad (4.5)$

7 Until 收敛

▷ 从数据集中抽取一张图片

▷ 从均匀分布中采样一个时间步

▷ 从正态分布中采样一个噪声

图 4.12 U-net 训练算法

官方的训练算法如上，下图是一个训练步骤的示意图：

针对每一个训练步骤 (step)

1. 随机选取一个时间步，然后对时间步进行编码



2. 给图片添加噪声



$$x_t = \sqrt{\bar{a}_t} x_0 + \sqrt{1 - \bar{a}_t} \varepsilon$$

根据时间步调整添加的噪声的数量

$$\varepsilon \sim \mathcal{N}(0, 1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

3. 训练UNet网络

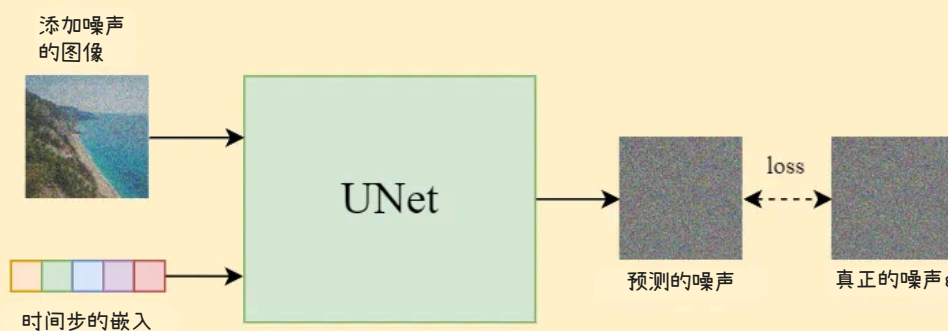


图 4.13 扩散模型训练步骤

4.2.4.3 反向扩散

```

1   $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$  ▷ 从高斯分布中采样一张纯噪声图片
2  for  $t = T, \dots, 1$  do
3       $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = 0$ 
4       $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(\mathbf{x}, t) \right) + \sigma_t \mathbf{z}$  ▷  $\sigma_t$ 一般取 $\sqrt{\beta_t}$ 
5  end for
6  return  $\mathbf{x}_0$ 

```

图 4.14 采样算法（去噪算法，反向扩散算法）

我们可以使用上述算法从噪声中生成图像。下图是它的说明：

反向扩散/去噪/采样

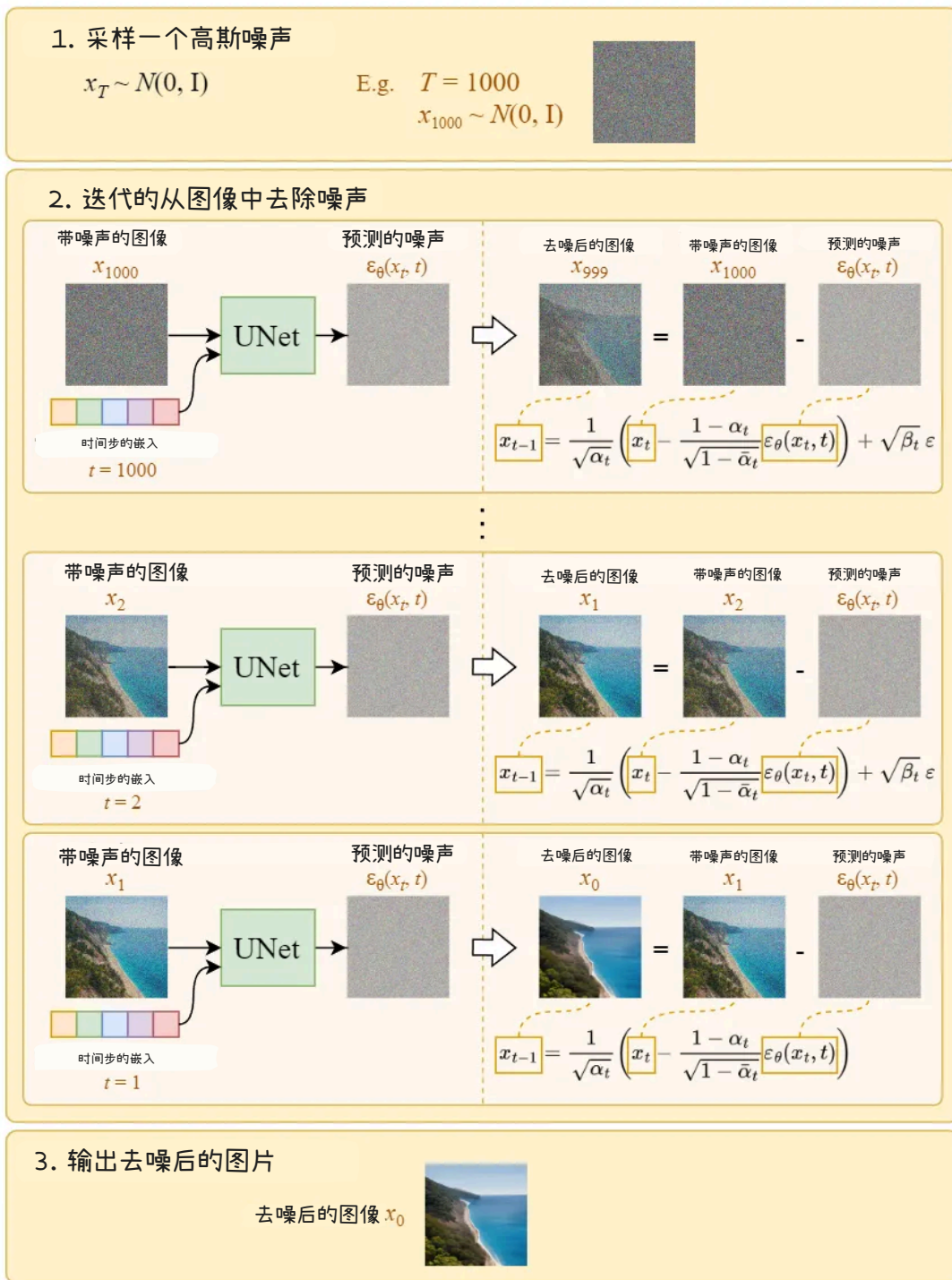


图 4.15 扩散模型的采样过程

请注意，在最后一步，我们只是输出学习到的平均值 $\mu_{\theta}(x_1, 1)$ ，而不向其中添加噪声。

4.3 代码实现

4.3.1 扩散过程相关代码

我们需要针对扩散的时间步来制定一个方差的调度计划。每个时间步，都在前面一个时间步的图像中添加一个高斯噪声。在开始向图像中添加噪声的时候，需要高斯噪声的方差小一些，不至于一开始就把图像变得很模糊。而到了后面，添加噪声就可以大胆一些了，反正已经模糊了。所以后面添加的高斯噪声需要方差大一些。

🔥 高斯噪声的方差

公式中的 β_t 就是第 t 个时间步要添加的噪声的方差。

在代码中如下

```
1 # self.betas: 方差计划调度表
2 self.betas = torch.linspace(
3     beta_start, # beta_start: 起始β值, 论文中等于0.0001
4     beta_end, # beta_end: 结束β值, 论文中等于0.02
5     num_timesteps, # num_timesteps: 时间步的数量: 1000
6     device=device
7 )
```

Python

而由于 $\alpha_t = 1 - \beta_t$ ，所以有如下代码

```
1 self.alphas = 1 - self.betas
```

Python

这样就计算出了每个时间步的 α_t 。

而由于 $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ，所以使用 `torch.cumprod` 来计算

```
1 self.alphaBars = torch.cumprod(self.alphas, dim=0)
```

Python

由于我们已经使用重参数技巧来给图像添加噪声，也就是通过解析解可以直接得到添加了 t 个时间步的噪声的图像。所以前向扩散过程就用了这个公式。

🔥 给定原始图片 x_0 和时间步 t 直接采样出 x_t 的公式

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad (4.6)$$

其中：

- $\bar{\alpha}_t = \alpha_t \alpha_{t-1} \dots \alpha_1 = \prod_{s=1}^t \alpha_s$
- $\alpha_t = 1 - \beta_t$
- $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$

公式和代码基本是对应的。

```
1 def add_noise(self, x_0, t):
2     # x_0是原始图片, t是时间步
3     # T: 时间步的数量
4     T = self.num_timesteps
5     # 确保 1 ≤ t ≤ T
6     assert (t ≥ 1).all() and (t ≤ T).all()
7
8     t_idx = t - 1 # alphaBars[0] is for t=1
```

Python

```

9      #  $\bar{\alpha}_t$ 
10     alpha_bar = self.alpha_bars[t_idx] # (N,)
11     N = alpha_bar.size(0)
12     alpha_bar = alpha_bar.view(N, 1, 1, 1) # (N, 1, 1, 1)
13     #  $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ 
14     # 噪声的形状或者点数等于图片的像素点
15     noise = torch.randn_like(x_0, device=self.device)
16     #  $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$ 
17     x_t = torch.sqrt(alpha_bar) * x_0 + torch.sqrt(1 - alpha_bar) * noise
18     # 返回第t步的图像 $x_t$ 和添加的噪声noise, 后面作为训练数据
19     return x_t, noise

```

接下来编写反向去噪过程的代码, 也就是从 x_t 预测 x_{t-1} 。

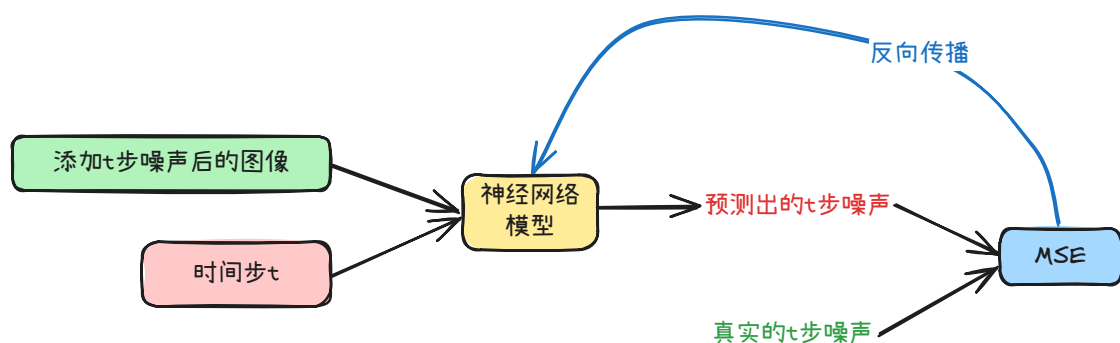


图 4.16 以噪声为预测目标

给定了时间步和添加噪声的图像, 模型可以预测出这幅图片中的噪声有多少, 那么我们从 x_t 中将预测出的噪声减掉, 就可以去噪了!

```

1  def denoise(self, model, x, t):
2      # x是图片, 模型会认为x是添加了t步噪声的图片
3      # t是时间步
4      T = self.num_timesteps
5      assert (t >= 1).all() and (t <= T).all()
6
7      t_idx = t - 1 # alphas[0] is for t=1
8      alpha = self.alphas[t_idx] #  $\alpha_t$ 
9      alpha_bar = self.alpha_bars[t_idx] #  $\bar{\alpha}_t$ 
10     alpha_bar_prev = self.alpha_bars[t_idx-1] #  $\bar{\alpha}_{t-1}$ 
11
12     N = alpha.size(0)
13     alpha = alpha.view(N, 1, 1, 1)
14     alpha_bar = alpha_bar.view(N, 1, 1, 1)
15     alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)
16
17     model.eval()
18     with torch.no_grad():
19         eps = model(x, t) # 根据时间步和图像x, 预测噪声
20         model.train()
21         # 公式中的z

```

```

22 noise = torch.randn_like(x, device=self.device)
23 # 如果时间步为1的话, 我们就不再添加 $\sigma_z$ 噪声
24 noise[t == 1] = 0 # no noise at t=1
25 # 均值 $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(x_t, t) \right)$ 
26 mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps)
27 / torch.sqrt(alpha)
28 # 标准差 $\sigma_t = \sqrt{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}$ 
29 std = torch.sqrt((1-alpha) * (1-alpha_bar_prev) \
30 / (1-alpha_bar))
31 # 返回 $x_{t-1}$ 
32 return mu + noise * std

```

```

1  $x_T \sim \mathcal{N}(0, \mathbf{I})$  ▷ 从高斯分布中采样一张纯噪声图片
2 for  $t = T, \dots, 1$  do
3      $z \sim \mathcal{N}(0, \mathbf{I})$  if  $t > 1$ , else  $z = 0$ 
4      $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(x, t) \right) + \sigma_t z$  ▷ 在 DDPM 论文中 $\sigma_t = \sqrt{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}$ 
5 end for
6 return  $x_0$ 

```

图 4.17 采样算法（去噪算法，反向扩散算法）

上面的代码实现了伪代码中的第 4 步。

4.3.2 时间步位置编码

由于训练网络需要时间步的信息，所以我们需要将时间步进行编码然后注入到网络中。

时间步信息通过正弦位置编码注入网络：

$$\mathbf{v}_i = \begin{cases} \sin\left(\frac{t}{10000^{\frac{i}{D}}}\right), & i \text{ 为偶数时} \\ \cos\left(\frac{t}{10000^{\frac{i}{D}}}\right), & i \text{ 为奇数时} \end{cases} \quad (4.7)$$

可以看到编码方式和 Transformer 中的几乎一样。

```

1 def _pos_encoding(time_idx, output_dim, device='cpu'):
2     t, D = time_idx, output_dim
3     v = torch.zeros(D, device=device)
4
5     i = torch.arange(0, D, device=device)
6     div_term = torch.exp(i / D * math.log(10000))
7     v[0::2] = torch.sin(t / div_term[0::2])
8     v[1::2] = torch.cos(t / div_term[1::2])
9     return v
10
11 def pos_encoding(timesteps, output_dim, device='cpu'):
12     batch_size = len(timesteps)

```

```

13     device = timesteps.device
14     v = torch.zeros(batch_size, output_dim, device=device)
15     for i in range(batch_size):
16         v[i] = _pos_encoding(timesteps[i], output_dim, device)
17     return v

```

4.3.3 U-Net 神经网络

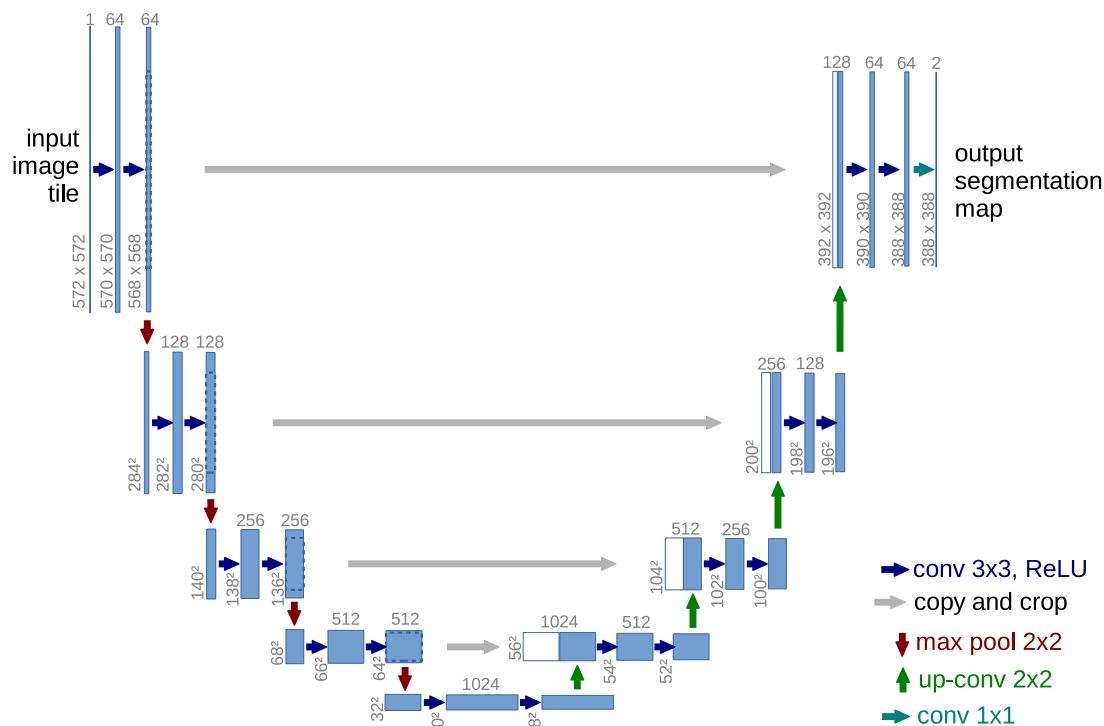


图 4.18 U-net 模型

先来编写卷积块。

```

1  class ConvBlock(nn.Module):
2      def __init__(self, in_ch, out_ch, time_embed_dim):
3          super().__init__()
4          self.convs = nn.Sequential(
5              nn.Conv2d(in_ch, out_ch, 3, padding=1),
6              nn.BatchNorm2d(out_ch),
7              nn.ReLU(),
8              nn.Conv2d(out_ch, out_ch, 3, padding=1),
9              nn.BatchNorm2d(out_ch),
10             nn.ReLU()
11         )
12         self.mlp = nn.Sequential(
13             nn.Linear(time_embed_dim, in_ch),
14             nn.ReLU(),
15             nn.Linear(in_ch, in_ch)
16         )
17

```

Python

```

18     def forward(self, x, v):
19         N, C, _, _ = x.shape
20         v = self.mlp(v)
21         v = v.view(N, C, 1, 1)
22         y = self.convs(x + v)
23         return y

```

完整的 U-Net 结果如下

```

1  class UNet(nn.Module): Python
2      def __init__(self, in_ch=1, time_embed_dim=100):
3          super().__init__()
4          self.time_embed_dim = time_embed_dim
5
6          self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
7          self.down2 = ConvBlock(64, 128, time_embed_dim)
8          self.bot1 = ConvBlock(128, 256, time_embed_dim)
9          self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
10         self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
11         self.out = nn.Conv2d(64, in_ch, 1)
12
13         self.maxpool = nn.MaxPool2d(2)
14         self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
15
16     def forward(self, x, timesteps):
17         """unet的输入: 图片x_t和时间步t"""
18
19         # 时间步的位置编码嵌入
20         v = pos_encoding(timesteps, self.time_embed_dim, x.device)
21
22         x1 = self.down1(x, v) # 下采样
23         x = self.maxpool(x1) # 最大池化
24         x2 = self.down2(x, v) # 下采样
25         x = self.maxpool(x2) # 最大池化
26
27         x = self.bot1(x, v)
28
29         x = self.upsample(x) # 上采样
30         x = torch.cat([x, x2], dim=1) # 跳跃连接
31         x = self.up2(x, v) # 上采样
32         x = self.upsample(x) # 上采样
33         x = torch.cat([x, x1], dim=1) # 跳跃连接
34         x = self.up1(x, v) # 上采样
35         x = self.out(x)
36         return x

```

4.4 完整代码

```

1  import math Python
2  import torch

```



```
3 import torchvision
4 import matplotlib.pyplot as plt
5 from torchvision import transforms
6 from torch.utils.data import DataLoader
7 from torch.optim import Adam
8 import torch.nn.functional as F
9 from torch import nn
10 from tqdm import tqdm
11
12
13 img_size = 28
14 batch_size = 128
15 num_timesteps = 1000
16 epochs = 10
17 lr = 1e-3
18 device = "cuda"
19
20
21 def show_images(images, rows=2, cols=10):
22     fig = plt.figure(figsize=(cols, rows))
23     i = 0
24     for r in range(rows):
25         for c in range(cols):
26             fig.add_subplot(rows, cols, i + 1)
27             plt.imshow(images[i], cmap='gray')
28             plt.axis('off')
29             i += 1
30     plt.show()
31
32 # 位置编码部分
33 def _pos_encoding(time_idx, output_dim, device='cpu'):
34     t, D = time_idx, output_dim
35     v = torch.zeros(D, device=device)
36
37     i = torch.arange(0, D, device=device)
38     div_term = torch.exp(i / D * math.log(10000))
39
40     v[0::2] = torch.sin(t / div_term[0::2])
41     v[1::2] = torch.cos(t / div_term[1::2])
42     return v
43
44 def pos_encoding(timesteps, output_dim, device='cpu'):
45     batch_size = len(timesteps)
46     device = timesteps.device
47     v = torch.zeros(batch_size, output_dim, device=device)
48     for i in range(batch_size):
49         v[i] = _pos_encoding(timesteps[i], output_dim, device)
50     return v
51
52 class ConvBlock(nn.Module):
```

```

53     def __init__(self, in_ch, out_ch, time_embed_dim):
54         super().__init__()
55         self.convs = nn.Sequential(
56             nn.Conv2d(in_ch, out_ch, 3, padding=1),
57             nn.BatchNorm2d(out_ch),
58             nn.ReLU(),
59             nn.Conv2d(out_ch, out_ch, 3, padding=1),
60             nn.BatchNorm2d(out_ch),
61             nn.ReLU()
62         )
63         self.mlp = nn.Sequential(
64             nn.Linear(time_embed_dim, in_ch),
65             nn.ReLU(),
66             nn.Linear(in_ch, in_ch)
67         )
68
69     def forward(self, x, v):
70         N, C, _, _ = x.shape
71         v = self.mlp(v)
72         v = v.view(N, C, 1, 1)
73         y = self.convs(x + v)
74         return y
75
76 class UNet(nn.Module):
77     def __init__(self, in_ch=1, time_embed_dim=100):
78         super().__init__()
79         self.time_embed_dim = time_embed_dim
80
81         self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
82         self.down2 = ConvBlock(64, 128, time_embed_dim)
83         self.bot1 = ConvBlock(128, 256, time_embed_dim)
84         self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
85         self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
86         self.out = nn.Conv2d(64, in_ch, 1)
87
88         self.maxpool = nn.MaxPool2d(2)
89         self.upsample = nn.Upsample(scale_factor=2, mode='bilinear')
90
91     def forward(self, x, timesteps):
92         v = pos_encoding(timesteps, self.time_embed_dim, x.device)
93
94         x1 = self.down1(x, v)
95         x = self.maxpool(x1)
96         x2 = self.down2(x, v)
97         x = self.maxpool(x2)
98
99         x = self.bot1(x, v)
100
101         x = self.upsample(x)
102         x = torch.cat([x, x2], dim=1)

```

```

103     x = self.up2(x, v)
104     x = self.upsample(x)
105     x = torch.cat([x, x1], dim=1)
106     x = self.up1(x, v)
107     x = self.out(x)
108     return x
109
110
111 class Diffuser:
112     def __init__(self,
113         num_timesteps=1000, # 时间步T=1000
114         beta_start=0.0001, # 方差的起始值 $\beta_0 = 0.0001$ 
115         beta_end=0.02, # 方差的最终值 $\beta_T = 0.02$ 
116         device="cpu"
117     ):
118         self.num_timesteps = num_timesteps
119         self.device = device
120         # 方差调度计划
121         self.betas = torch.linspace(
122             beta_start,
123             beta_end,
124             num_timesteps,
125             device=device
126         )
127         #  $\alpha_t = 1 - \beta_t$ 
128         self.alphas = 1 - self.betas
129         #  $\bar{\alpha}_t = \alpha_t \alpha_{t-1} \cdots \alpha_1$ 
130         self.alpha_bars = torch.cumprod(self.alphas, dim=0)
131
132     def add_noise(self, x_0, t):
133         T = self.num_timesteps
134         assert (t >= 1).all() and (t <= T).all()
135
136         t_idx = t - 1 # alpha_bars[0] is for t=1
137         alpha_bar = self.alpha_bars[t_idx] # (N,)
138         N = alpha_bar.size(0)
139         alpha_bar = alpha_bar.view(N, 1, 1, 1) # (N, 1, 1, 1)
140
141         noise = torch.randn_like(x_0, device=self.device)
142         x_t = torch.sqrt(alpha_bar) * x_0 \
143             + torch.sqrt(1 - alpha_bar) * noise
144         return x_t, noise
145
146     def denoise(self, model, x, t):
147         """去除一步噪声, x为时间步t的带噪声图片 $x_t$ """
148         T = self.num_timesteps
149         assert (t >= 1).all() and (t <= T).all()
150
151         t_idx = t - 1 # alphas[0] is for t=1
152         alpha = self.alphas[t_idx]

```

```

153     alpha_bar = self.alpha_bars[t_idx]
154     alpha_bar_prev = self.alpha_bars[t_idx-1]
155
156     N = alpha.size(0)
157     alpha = alpha.view(N, 1, 1, 1)
158     alpha_bar = alpha_bar.view(N, 1, 1, 1)
159     alpha_bar_prev = alpha_bar_prev.view(N, 1, 1, 1)
160
161     model.eval()
162     with torch.no_grad():
163         eps = model(x, t)
164     model.train()
165
166     noise = torch.randn_like(x, device=self.device)
167     noise[t == 1] = 0 # no noise at t=1
168
169     mu = (x - ((1-alpha) / torch.sqrt(1-alpha_bar)) * eps) \
170         / torch.sqrt(alpha)
171     std = torch.sqrt(
172         (1-alpha) * (1-alpha_bar_prev) / (1-alpha_bar)
173     )
174     #  $x_{t-1}$ 
175     return mu + noise * std
176
177     def reverse_to_img(self, x):
178         x = x * 255
179         x = x.clamp(0, 255)
180         x = x.to(torch.uint8)
181         x = x.cpu()
182         to_pil = transforms.ToPILImage()
183         return to_pil(x)
184
185     def sample(self, model, x_shape=(20, 1, 28, 28)):
186         """从纯噪声图片 $x_{1000}$ 反向扩散出 $x_0$ """
187         batch_size = x_shape[0]
188         # 采样一张白噪声图片 $x_{1000}$ 出来
189         x = torch.randn(x_shape, device=self.device)
190         # for t = T, T-1, ..., 0
191         for i in tqdm(range(self.num_timesteps, 0, -1)):
192             t = torch.tensor(
193                 [i] * batch_size,
194                 device=self.device,
195                 dtype=torch.long
196             )
197             # 一步去噪,  $x_t \rightarrow x_{t-1}$ 
198             x = self.denoise(model, x, t)
199
200         images = [
201             self.reverse_to_img(x[i])
202             for i in range(batch_size)

```

```

203     ]
204     return images
205
206
207 preprocess = transforms.ToTensor()
208 dataset = torchvision.datasets.MNIST(
209     root="./../datasets",
210     download=True,
211     transform=preprocess
212 )
213 dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
214
215 diffuser = Diffuser(num_timesteps, device=device)
216 model = UNet()
217 model.to(device)
218 optimizer = Adam(model.parameters(), lr=lr)
219
220 losses = []
221 for epoch in range(epochs):
222     loss_sum = 0.0
223     cnt = 0
224
225     # 每个 epoch 都生成采样的图像 =====
226     images = diffuser.sample(model)
227     show_images(images)
228     # =====
229
230     for images, labels in tqdm(dataloader):
231         optimizer.zero_grad()
232         x = images.to(device)
233         # 随机取一个时间步
234         t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
235         # x_noisy是 $x_t$ , noise是添加的真正的噪声
236         x_noisy, noise = diffuser.add_noise(x, t)
237         # 模型根据 $x_t$ 和时间步 $t$ , 预测给 $x_t$ 添加的噪声
238         noise_pred = model(x_noisy, t)
239         # 添加的真实噪声和预测噪声之间进行均方误差计算
240         loss = F.mse_loss(noise, noise_pred)
241
242         loss.backward()
243         optimizer.step()
244
245         loss_sum += loss.item()
246         cnt += 1
247
248     loss_avg = loss_sum / cnt
249     losses.append(loss_avg)
250     print(f"Epoch {epoch} | Loss: {loss_avg}")
251
252 # 画出损失

```

```
253 plt.plot(losses)
254 plt.xlabel("Epoch")
255 plt.ylabel("Loss")
256 plt.show()
257
258 # 从完全噪声的图片反向扩散
259 images = diffuser.sample(model)
260 show_images(images)
```


5. 条件扩散模型

我们之前对数据 x 的概率 $p(x)$ 进行了建模。但在实用层面，我们更希望对条件概率 $p(x|y)$ 进行建模（其中 y 表示条件）。如果能成功建立 $p(x|y)$ 的模型，那么就可以通过条件 y 控制想生成的 x 。

条件 y 可以是文本、图像或标签等。如果 y 是一张低分辨率的图像，那么可以考虑将其变换为高分辨率的图像。这就是被称为超分辨率成像（**super-resolution imaging**）的技术。使用扩散模型进行超分辨率处理的模型包括级联扩散模型（**cascaded diffusion model**）等。

本节将以 y 作为标签的例子进行说明。具体来说，我们将创建一个模型，在给定 MNIST 数据的数字标签后，该模型能够生成与该标签相对应的图像。

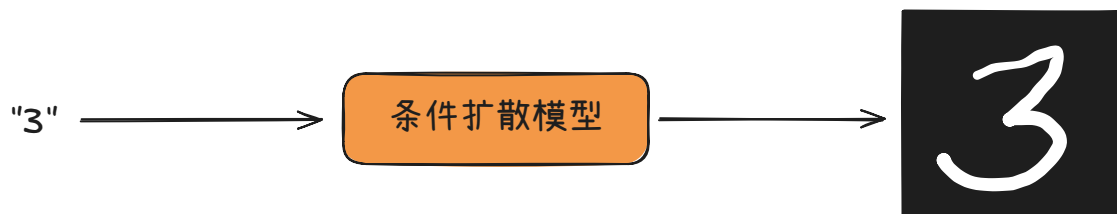


图 5.1 本章将要实现的条件扩散模型

5.1 向扩散模型添加条件

首先，我们从复习开始。在扩散模型中，我们可以选择在何处使用神经网络。例如，可以考虑使用神经网络对 $\mu_{\theta}(x_t, t)$ 和 $\epsilon_{\theta}(x_t, t)$ 进行建模。

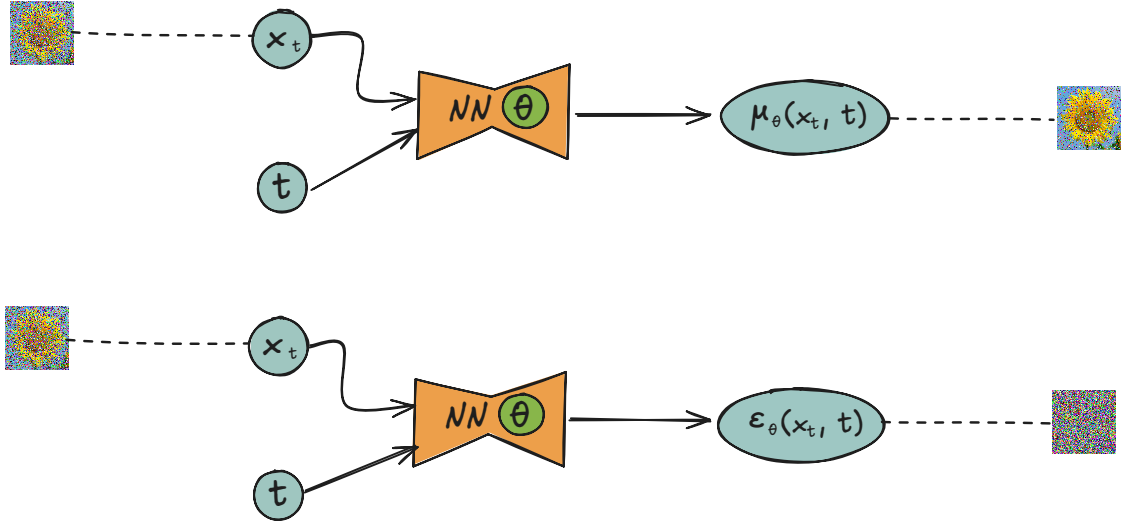


图 5.2 预测时刻 $t-1$ 的图像（正态分布的均值向量）的模型（上）和预测添加到 x_t 中的噪声的模型（下）

下面思考使用神经网络对 $\mu_\theta(x_t, t)$ 建模的情况。在这种情况下， $p_\theta(x_{t-1}|x_t)$ 的数学式如下所示。

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_q^2(t)\mathbf{I}) \quad (5.1)$$

数据 x_0 的概率 $p_\theta(x_0)$ 的数学式如下所示：

$$\begin{aligned} p_\theta(x_0) &= \int p_\theta(x_0, x_1, \dots, x_T) dx_1 \cdots dx_T && \text{（概率的边际化）} \\ &= \int p_\theta(x_0|x_1) \cdots p_\theta(x_{T-1}|x_T) p(x_T) dx_1 \cdots dx_T && \text{（马尔可夫性质）} \end{aligned} \quad (5.2)$$

这里有 $p(x_T) = \mathcal{N}(x_T; \mathbf{0}, \mathbf{I})$ 。

为了得到条件扩散模型，我们要建模的对象是 $p_\theta(x_0|y)$ ，而不是 $p_\theta(x_0)$ 。实现这个目标的最简单的方法是在每个时刻的 $p_\theta(x_{t-1}|x_t)$ 中添加条件 y 。具体的数学式如下所示：

$$p_\theta(x_0|y) = \int p_\theta(x_0|x_1, y) \cdots p_\theta(x_{T-1}|x_T, y) p(x_T) dx_1 dx_T \quad (5.3)$$

此时的 $p_\theta(x_{t-1}|x_t, y)$ 的数学式如下所示：

$$p_\theta(x_{t-1}|x_t, y) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t, y), \sigma_q^2(t)\mathbf{I}) \quad (5.4)$$

在常规的扩散模型中， $\mu_\theta(x_t, t)$ 通常由神经网络进行建模。而在条件扩散模型中，如 $\mu_\theta(x_t, t, y)$ 所示，会额外添加参数 y 。换言之，通过向神经网络中添加 y ，可以使模型“进化”为条件扩散模型。同样，对于预测噪声的神经网络，也可以通过将参数 y 添加到 $\epsilon_\theta(x_t, t)$ 中，使其变为 $\epsilon_\theta(x_t, t, y)$ 来实现相应的功能（参见图 5.3）。

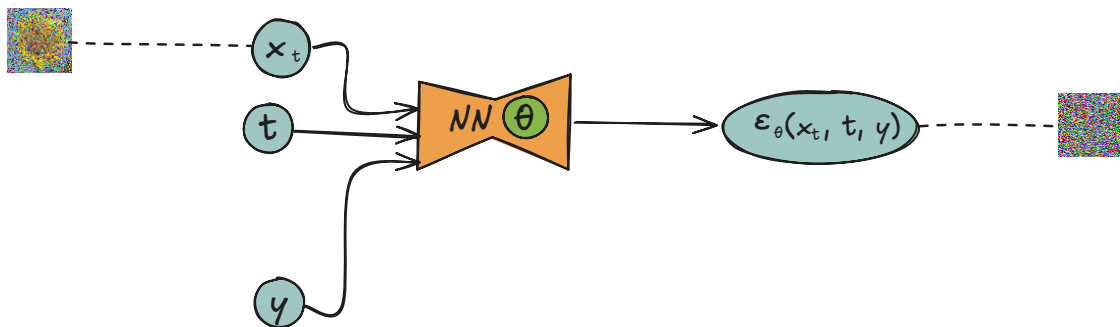


图 5.3 条件扩散模型的神经网络

5.2 条件扩散模型的实现

我们已经使用神经网络实现了 $\epsilon_{\theta}(x_t, t)$ 。 $\epsilon_{\theta}(x_t, t)$ 的参数 t 是整数，可以通过正弦位置编码变换为向量。而新添加的条件 y 是标签，也是整数。这个整数 y 可以通过嵌入层 (embedding layer) 变换为向量。具体来说， y 被变换成向量，然后与变换后的 t 向量相加。

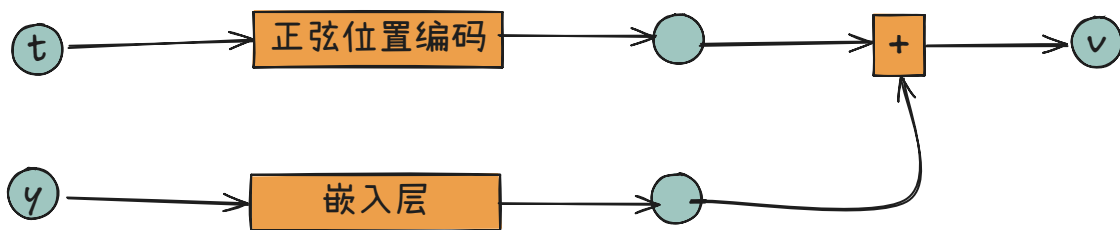


图 5.4 在扩散模型中添加嵌入层

提示

嵌入层的初始值被设置为随机值，然后通过训练进行优化。这样就可以训练得到与每个标签相对应的匹配任务的向量。而由于与时刻 t 相关的特定任务的训练要素很少，因此我们使用了一种称为正弦位置编码的固定向量变换的方法。

以下是代码示例。这里我们在上一章中实现的 UNet 类中添加了名为 UNetCond 类的代码，类名中的 Cond 是 Conditional (条件) 的缩写。

```

1 class UNetCond(nn.Module):
2     def __init__(self, in_ch=1, time_embed_dim=100, num_labels=None):
3         super().__init__()
4         self.time_embed_dim = time_embed_dim
5
6         self.down1 = ConvBlock(in_ch, 64, time_embed_dim)
7         self.down2 = ConvBlock(64, 128, time_embed_dim)
8         self.bot1 = ConvBlock(128, 256, time_embed_dim)
9         self.up2 = ConvBlock(128 + 256, 128, time_embed_dim)
10        self.up1 = ConvBlock(128 + 64, 64, time_embed_dim)
11        self.out = nn.Conv2d(64, in_ch, 1)
12
13        self.maxpool = nn.MaxPool2d(2)

```

```

14         self.upsample = nn.Upsample(scale_factor=2, mode="bilinear")
15
16         # ① 处理标签的嵌入层
17         if num_labels is not None:
18             self.label_emb = nn.Embedding(num_labels, time_embed_dim)
19
20     def forward(self, x, timesteps, labels=None):
21         t = pos_encoding(timesteps, self.time_embed_dim)
22
23         # ② 标签的处理
24         if labels is not None:
25             t += self.label_emb(labels)
26
27         x1 = self.down1(x, t)
28         x = self.maxpool(x1)
29         x2 = self.down2(x, t)
30         x = self.maxpool(x2)
31
32         x = self.bot1(x, t)
33
34         x = self.upsample(x)
35         x = torch.cat([x, x2], dim=1)
36         x = self.up2(x, t)
37         x = self.upsample(x)
38         x = torch.cat([x, x1], dim=1)
39         x = self.up1(x, t)
40         x = self.out(x)
41         return x

```

代码的主要修改有两处。在代码①处，我们准备了处理标签的嵌入层（`nn.Embedding`）。具体来说，就是通过 `nn.Embedding(num_labels, time_embed_dim)` 将共计 `num_labels` 个不同整数变换为 `time_embed_dim` 维的向量。在代码②处，通过 `nn.Embedding` 层处理标签，然后再加到变换后的时间数据的向量中。

接下来是 `Diffuser` 类。对于 `Diffuser` 类，修改生成数据的方法。下面只展示了修改部分的代码。

```

1  class Diffuser: Python
2      def denoise(self, model, x, t, labels):
3          # ... (省略)
4          with torch.no_grad():
5              eps = model(x, t, labels) # 同时提供labels
6              # ...
7
8      def sample(self, model, x_shape=(20, 1, 28, 28), labels=None):
9          # ...
10         if labels is None:
11             labels = torch.randint(0, 10, (len(x),), device=self.device)
12         # ...
13         for i in tqdm(range(self.num_timesteps, 0, -1)):
14             t = torch.tensor([i] * batch_size, device=self.device,
15                             dtype=torch.long)
16             x = self.denoise(model, x, t, labels) # 还提供labels

```

```

17         # ...
18         return images, labels

```

修改的部分还向模型提供了标签。具体来说，标签是在生成数据和去噪过程中提供的。最后是训练的代码。

```

1  diffuser = Diffuser(num_timesteps, device=device)
2  model = UNetCond(num_labels=10) ❶
3  model.to(device)
4  optimizer = Adam(model.parameters(), lr=lr)
5
6  losses = []
7  for epoch in range(epochs):
8      loss_sum = 0.0
9      cnt = 0
10
11     for images, labels in tqdm(dataloader):
12         optimizer.zero_grad()
13         x = images.to(device) ❷
14         t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
15
16         x_noisy, noise = diffuser.add_noise(x, t)
17         noise_pred = model(x_noisy, t, labels) ❸
18         loss = F.mse_loss(noise, noise_pred)
19
20         loss.backward()
21         optimizer.step()
22
23         loss_sum += loss.item()
24         cnt += 1

```

训练代码的修改内容如下所示。

❶ UNetCond(num_labels=10)

创建一个具有 10 个分类的条件扩散模型。

❷ labels.to(device)

在 device 上准备标签数据。

❸ model(x_noisy, t, labels)

向模型提供 labels 进行训练。

以上就是条件扩散模型的实现。现在运行代码。经过 10 轮训练后，最终生成了下图所示的图像。

虽然仍有改进的余地，但生成的图像是符合条件的。在下一节中，我们将探讨进一步改进这个条件扩散模型的方法。

5.3 得分函数

在上面的内容里面，我们实现了一个简单的条件扩散模型。这个“简单”的意思是我们只是向模型提供了条件而已。因此，模型有可能会轻视我们提供的条件，甚至在最坏的情况下有可能会忽略我们提供的条件。接下来，我们将介绍一种称为“指引”的方法。指引是一种将给定条件纳入模型并给予更多重视的机制。

要理解指引机制，首先需要了解得分函数。

5.3.1 什么是得分函数

在实现扩散模型时，我们使用神经网络对 $\varepsilon_{\theta}(x_t, t)$ 进行了建模。 $\varepsilon_{\theta}(x_t, t)$ 基于 x_t 和 t 来推断噪声 ε 。下图是这个过程的示意图。

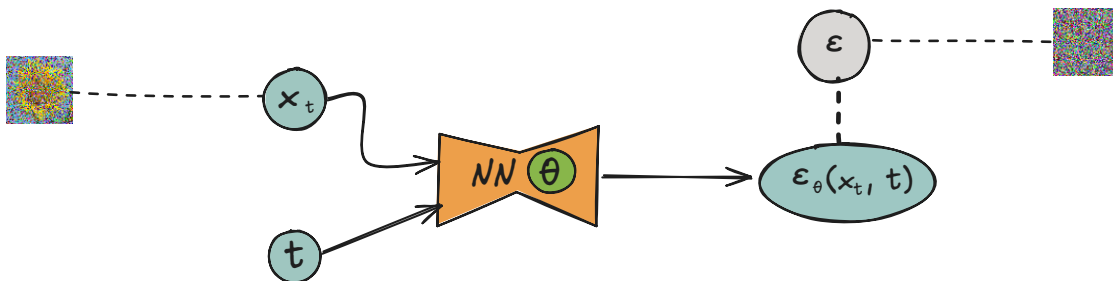


图 5.5 扩散模型中使用的推断噪声 ε 的神经网络

此时有以下与 ε 相关的数学式成立（稍后将给出证明）。

$$\varepsilon \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p(x_t) \quad (5.5)$$

∇ 是表示梯度的符号，读作“那勃乐”（Nabla）。 $\nabla_{x_t} \log p(x_t)$ 是对数似然 $\log p(x_t)$ 对输入数据 x_t 的梯度，称为“得分函数”或“得分”。

提示

$p(x_t)$ 是表示数据 x_t 为“真”的随机密度函数。而 $p_{\theta}(x_t)$ 表示使用参数对真的概率密度函数进行近似的概率密度函数。由于 $\nabla_{x_t} \log p(x_t)$ 和 $\nabla_{x_t} \log p_{\theta}(x_t)$ 表示关于输入 x_t 的梯度，因此称为“得分”。另外，在某些领域，关于参数的梯度（ $\nabla_{\theta} \log p_{\theta}(x_t)$ ）也称为“得分”。在本书中，我们将对输入的梯度称为“得分”。

根据式（5.5）， ε 可以近似表示为 $\nabla_{x_t} \log p(x_t)$ 。值得注意的是， ε 与 $\nabla_{x_t} \log p(x_t)$ 之间只相差负常数倍（ $-\sqrt{1 - \bar{\alpha}_t}$ ）。这说明以 $-\sqrt{1 - \bar{\alpha}_t} \varepsilon$ 来代替 ε 作为训练数据的神经网络也是可行的。

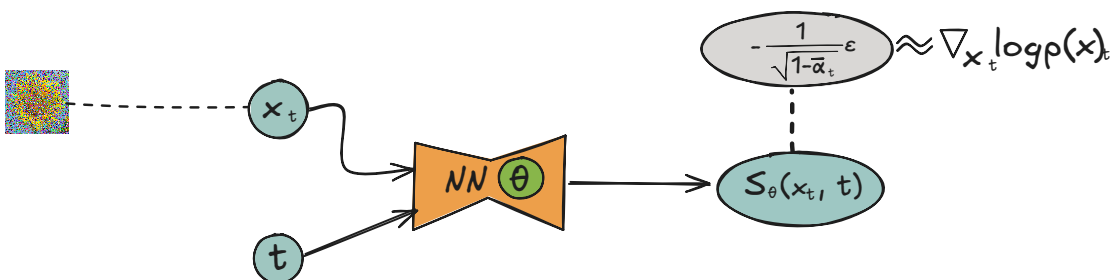


图 5.6 用于推断得分的神经网络

图中的神经网络将得分的近似值 $-\sqrt{1 - \bar{\alpha}_t} \varepsilon$ 推断为 $s_{\theta}(x_t, t)$ 。因此，我们也可以通过推断得分的神经网络来实现扩散模型。

🔥 提示

生成模型中包括对得分进行建模的方法。这些模型统称为基于得分的模型。以对数似然最大化的方式可以推导出各种模型 (GMM、VAE、扩散模型等)。这些模型可以被称为基于似然的模型。重要的是, 扩散模型也可以作为基于得分的模型推导得出。

5.3.2 式 (5.5) 的证明

现在让我们来证明 式 (5.5) 成立。

$$\varepsilon \approx -\sqrt{1-\bar{\alpha}_t} \nabla_{x_t} \log p(x_t) \quad (5.6)$$

我们先来复习一下。时刻 t 的噪声数据 x_t 可以基于以下正态分布从原始数据 x_0 生成。

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}) \quad (5.7)$$

利用重参数化技巧, 可以通过以下式子得到 x_t 的样本。

$$\begin{aligned} \varepsilon &\sim \mathcal{N}(0, \mathbf{I}) \\ x_t &= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon \end{aligned} \quad (5.8)$$

接下来要用到的是 **Tweedie** 公式, 这个公式如下所示。

🔥 Tweedie 公式

当基于 $x \sim \mathcal{N}(x; \mu, \Sigma)$ 得到样本 x 时, 有以下式子成立。

$$\mathbb{E}[\mu|x] = x + \Sigma \nabla_x \log p(x) \quad (5.9)$$

式子中的 μ 被视为随机变量。左侧的 $\mathbb{E}[\mu|x]$ 是在 x 作为条件下的 μ 的期望值。右侧的 $\nabla_x \log p(x)$ 表示得分。我们将这个 **Tweedie** 公式应用于式 (5.7)。

🔥 Tweedie 公式的应用

当基于 $x_t \sim \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$ 得到样本 x_t 时, 有以下式子成立。

$$\begin{aligned} \mathbb{E}[\sqrt{\bar{\alpha}_t}x_0|x_t] &= x_t + (1-\bar{\alpha}_t)\mathbf{I} \nabla_{x_t} \log p(x_t) \\ &= x_t + (1-\bar{\alpha}_t) \nabla_{x_t} \log p(x_t) \end{aligned} \quad (5.10)$$

然后, 利用 式 (5.8) 中的 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$ 成立这一事实将式子展开, 如下所示。

$$\begin{aligned} \mathbb{E}[\sqrt{\bar{\alpha}_t}x_0|x_t] &= x_t + (1-\bar{\alpha}_t) \nabla_{x_t} \log p(x_t) \\ \Leftrightarrow \mathbb{E}[x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon|x_t] &= x_t + (1-\bar{\alpha}_t) \nabla_{x_t} \log p(x_t) \\ \Leftrightarrow \mathbb{E}[x_t|x_t] - \mathbb{E}[\sqrt{1-\bar{\alpha}_t}\varepsilon|x_t] &= x_t + (1-\bar{\alpha}_t) \nabla_{x_t} \log p(x_t) \\ \Leftrightarrow x_t - \sqrt{1-\bar{\alpha}_t}\mathbb{E}[\varepsilon|x_t] &= x_t + (1-\bar{\alpha}_t) \nabla_{x_t} \log p(x_t) \\ \therefore \mathbb{E}[\varepsilon|x_t] &= (1-\bar{\alpha}_t) \nabla_{x_t} \log p(x_t) \end{aligned} \quad (5.11)$$

期望值 $\mathbb{E}[\varepsilon|x_t]$ 可以用蒙特卡洛方法近似。这里我们用一个采样数据来近似 x_t 。这样就得到了以下数学式。

$$\varepsilon \approx -\sqrt{1-\bar{\alpha}_t} \nabla_{x_t} \log p(x_t) \quad (5.12)$$

这样我们就完成了推导。

5.4 分类器指引

在上面的内容中，我们了解到可以通过预测得分的神经网络来实现扩散模型。在本节中，我们将从得分的角度来研究扩散模型，并推导出一种称为指引的方法。指引有两种主要类型：分类器指引（**classifier guidance**）和无分类器指引（**classifier-free guidance**）。我们先来介绍分类器指引。

5.4.1 什么是分类器

分类器指引是一种使用分类器指导数据生成的方法。分类器是一个能够对数据进行分类的预训练神经网络，如图所示。

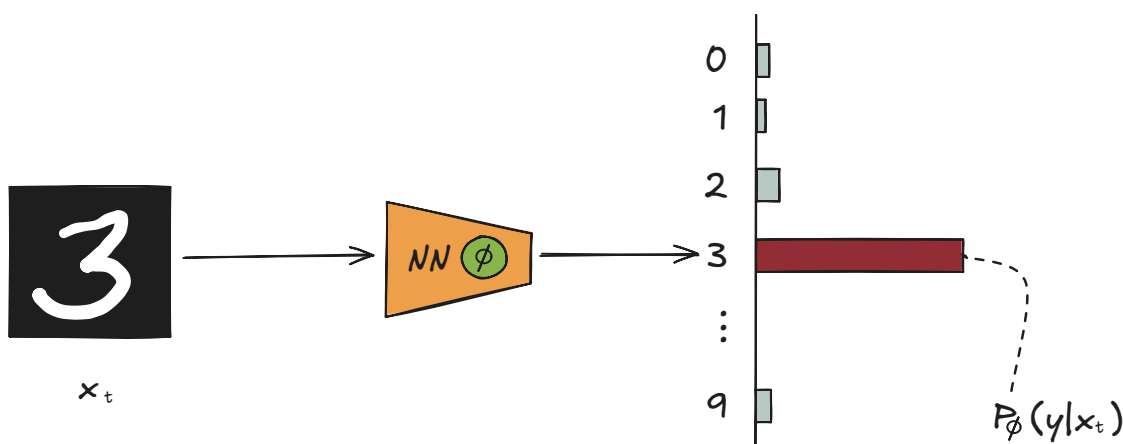


图 5.7 使用预训练神经网络进行分类的示例（参数为 ϕ ）

图中的神经网络将时刻 t 的噪声图像 x_t 作为输入，并输出类别 y 的概率 $p_\phi(y|x_t)$ 。利用这个分类器的神经网络，可以将普通的扩散模型变换为条件扩散模型。在这个过程中，我们需要掌握以下两点。

- 扩散模型可以通过预测得分 $\nabla_{x_t} \log p(x_t)$ 的神经网络来实现。
- （基于同样的原理）条件扩散模型可以通过预测条件概率得分 $\nabla_{x_t} \log p(x_t|y)$ 的神经网络来实现。

在记住以上两点之后，我们进行式子的变形，推导出分类器指引。

5.4.2 分类器指引的推导

首先，根据贝叶斯定理，将条件概率 $p(x_t|y)$ 表示为下面的数学式。

$$p(x_t|y) = \frac{p(x_t)p(y|x_t)}{p(y)} \quad (5.13)$$

然后，计算关于 x_t 的梯度（ ∇_{x_t} ）。

$$\begin{aligned} \nabla_{x_t} \log p(x_t|y) &= \nabla_{x_t} \log \left(\frac{p(x_t)p(y|x_t)}{p(y)} \right) \\ &= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t) - \underbrace{\nabla_{x_t} \log p(y)}_0 \\ &= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t) \end{aligned} \quad (5.14)$$

中间的式子中出现 $\nabla_{x_t} \log p(y)$ ，但由于 $p(y)$ 中不包含 x_t ，因此 $\nabla_{x_t} \log p(y) = 0$ 。由此得到的数学式如下所示。

$$\underbrace{\nabla_{x_t} \log p(x_t|y)}_{\text{条件得分}} = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{①得分}} + \underbrace{\nabla_{x_t} \log p(y|x_t)}_{\text{②分类器的对数似然的梯度}} \quad (5.15)$$

根据这个式子可以推导出分类器指引。下面是对式子中的要点的说明。

- ① 此项可以使用预测得分的神经网络 $s_\theta(x_t, t)$ 来计算。
- ② 此项可以使用作为分类器的神经网络来计算。

由此可见,我们可以利用得分和分类器这两个神经网络来表示条件得分(而一旦有了条件得分,便可以实现条件扩散模型)。另外,②的 $\nabla_{x_t} \log p(y|x_t)$ 可以很容易地通过反向传播求出。

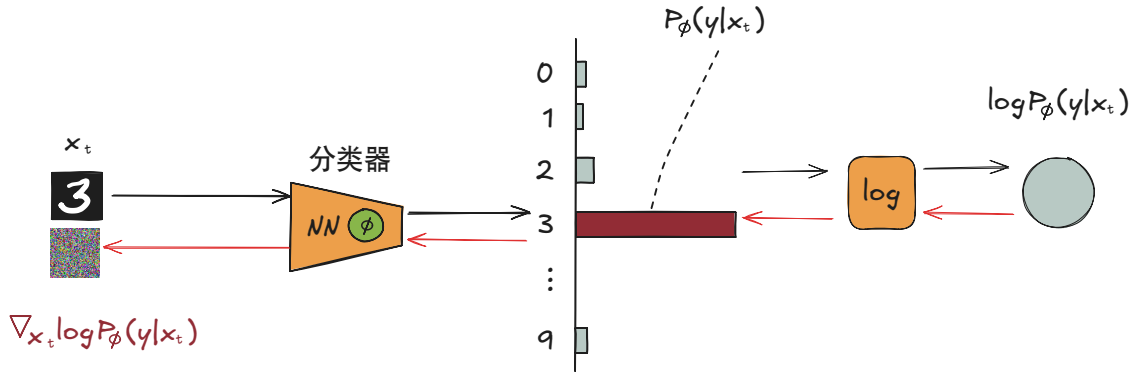


图 5.8 通过反向传播可以求出 $\nabla_{x_t} \log p(y|x_t)$

上图中的 $\nabla_{x_t} \log p(y|x_t)$ 显示了在当前 x_t 下,类别 y 的对数似然增加最快的方向。因此,如果沿着 $\nabla_{x_t} \log p(y|x_t)$ 的方向更新 x_t ,那么更新后的图像被分类为类别 y 的概率会更高。

分类器指引的理念是使用得分和分类器来表示条件得分。通常,我们会向分类器指引中引入权重 γ ,这样就能调整分类器的贡献度。引入后的数学式如下所示。

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log p(y|x_t) \quad (5.16)$$

权重 γ 是人为设定的值(超参数),用于调整分类器向类别 y 方向引导的程度。 γ 值越大,条件 y 的作用就越显著。使用表示推断得分的神经网络 $s_\theta(x_t, t)$ 和表示分类器的 $p_\phi(y|x_t)$,可以将式子改写为如下形式。

$$\nabla_{x_t} \log p(x_t|y) \approx s_\theta(x_t, t) + \gamma \nabla_{x_t} \log p(y|x_t) \quad (5.17)$$

作为参考,上式所执行的处理如下图所示。

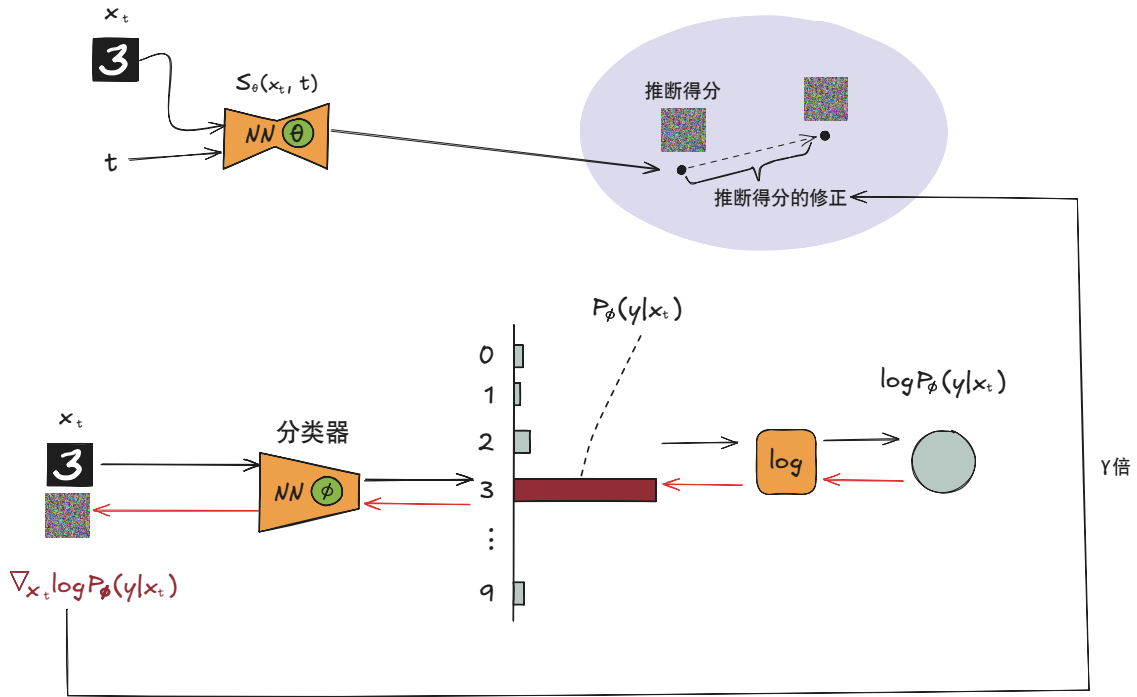


图 5.9 使用了推断得分的神经网络的分类器指引

如图所示，分类器可以与常规的扩散模型（推断得分的模型）相结合，作为条件扩散模型来生成数据。以上就是分类器指引的核心思想。

5.5 无分类器指引

分类器指引可以生成强调条件的数据。然而，分类器指引在实际应用中存在一个问题，即需要单独准备一个分类器。针对分类器指引的这一缺点进行改进的技术是无分类器指引。

5.5.1 无分类器指引的理论知识

顾名思义，无分类器指引就是不需要分类器的指引。它的机制可以通过以下过程得出。

$$\begin{aligned}
 & \nabla_{x_t} \log p(x_t|y) \\
 &= \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log p(y|x_t) \\
 &= \nabla_{x_t} \log p(x_t) + \gamma \nabla_{x_t} \log \frac{p(x_t|y)p(y)}{p(x_t)} \\
 &= \nabla_{x_t} \log p(x_t) + \gamma \left(\nabla_{x_t} \log p(x_t|y) + \underbrace{\nabla_{x_t} \log p(y) - \nabla_{x_t} \log p(x_t)}_0 \right) \\
 &= \nabla_{x_t} \log p(x_t) + \gamma (\nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t))
 \end{aligned} \tag{5.18}$$

上面我们使用贝叶斯定理进行了式子的展开，最后得到了上面的式子。根据上面的式子，可以推导出无分类器指引。上式的意思是从点 $\nabla_{x_t} \log p(x_t)$ 出发，沿着 $\nabla_{x_t} \log p(x_t|y)$ 的方向前进了 γ 倍的距离。下图是这个式子的示意图。

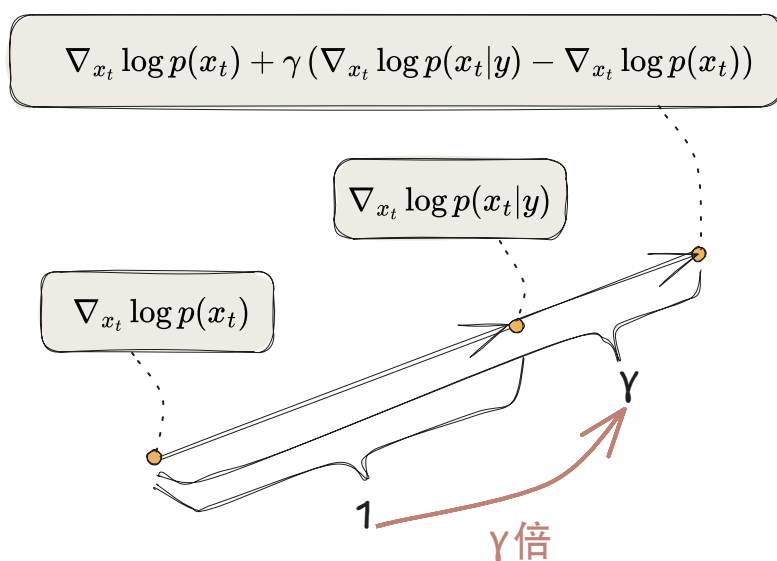


图 5.10 上面式子的可视化

式子中的 $\nabla_{x_t} \log p(x_t)$ 和 $\nabla_{x_t} \log p(x_t|y)$ 分别表示无条件得分和条件得分。它们可以由以下两个神经网络进行推断。

- 无条件得分推断器: $s_{\theta_1}(x_t, t)$
- 条件得分的推断器: $s_{\theta_2}(x_t, t, y)$

不过，准备两个模型很麻烦。更简单的方法是使用一个条件得分推断器 $s_{\theta_2}(x_t, t, y)$ ，并按如下方式建模。

- 无条件得分的推断器: $s_{\theta}(x_t, t, \emptyset)$
- 条件得分的推断器: $s_{\theta}(x_t, t, y)$

这里我们用 $\emptyset$ 表示“无条件”对应的类别。条件 y 可以被嵌入层变换为向量，但当类别为 $\emptyset$ 时，它会被变换为“零向量”，使其不包含任何信息。根据以上信息，无分类器指引的数学式可以表示成如下形式。

$$\nabla_{x_t} \log p(x_t|y) \approx s_{\theta}(x_t, t, \emptyset) + \gamma(s_{\theta}(x_t, t, y) - s_{\theta}(x_t, t, \emptyset)) \quad (5.19)$$

下图展示了这一计算过程。

上面介绍的是推断得分函数的神经网络，但由于得分和噪声之间只有常数倍的差别，因此同样的机制也适用于推断噪声的神经网络。

🔥 提示

在提供文本的图像生成服务中，我们经常会用到一种名为反向提示词（**negative prompt**）的技术。反向提示词是一种指定不希望生成的文本的技术。在上面的公式的 $\emptyset$ 中插入反向提示词即可实现这一技术。

5.5.2 无分类器指引的实现

下面我们通过修改节中的部分代码来实现无分类器指引。先回忆一下前面已经实现的 `UNetCond` 类，然后在这个类中实现 `forward()` 方法，代码如下所示。

```
1 class UNetCond(nn.Module):
```

Python

```

2     #... (省略)
3     def forward(self, x, timesteps, labels=None):
4         t = pos_encoding(timesteps, self.time_embed_dim)
5
6         if labels is not None:
7             t += self.label_emb(labels)
8         #...

```

从上面的代码来看，只有指定了参数 `labels`，才会处理类标签。如果没有指定参数（当 `labels=None` 时），则不做任何处理。这相当于“无条件”的处理。

无分类器指引的训练是在“无条件”和“有条件”两种情况下进行的条件扩散模型的训练。例如，我们可以以一定的比例进行“无条件”的训练，除此之外进行“有条件”的训练。基于这一点，我们可以按如下方式实现。

```

1  for epoch in range(epochs):
2      loss_sum = 0.0
3      cnt = 0
4
5      for images, labels in tqdm(dataloader):
6          optimizer.zero_grad()
7          x = images.to(device)
8          labels = labels.to(device)
9          t = torch.randint(1, num_timesteps+1, (len(x),), device=device)
10
11         # 以10%的概率进行"无条件"的训练
12         if np.random.random() < 0.1:
13             labels = None
14
15         x_noisy, noise = diffuser.add_noise(x, t)
16         noise_pred = model(x_noisy, t, labels)
17         loss = F.mse_loss(noise, noise_pred)
18         loss.backward()
19         optimizer.step()
20
21         loss_sum += loss.item()
22         cnt += 1

```

上面的代码以 10% 的概率进行了“无条件”的训练。对于“无条件”的情况，设置 `labels=None`。最后是 `Diffuser` 类进行去噪处理的代码。

```

1  class Diffuser:
2      #... (省略)
3
4      def denoise(self, model, x, t, labels, gamma):
5          #...
6          with torch.no_grad():
7              eps_cond = model(x, t, labels)
8              eps_uncond = model(x, t)
9              eps = eps_uncond + gamma * (eps_cond - eps_uncond)
10         #...

```

参数 `gamma` 的值越大，就越应该重视条件部分。另外，这里实现的是如下数学式的代码。

$$\varepsilon \approx \varepsilon_{\theta}(x_t, t, \emptyset) + \gamma(\varepsilon_{\theta}(x_t, t, y) - \varepsilon_{\theta}(x_t, t, \emptyset)) \quad (5.20)$$

经过以上修改之后，我们就完成了无分类器指引的实现。这里以 $\gamma = 3$ （`gamma=3.0`）声称了数据。

无分类器指引可以通过 γ 来调整条件的重视程度。而且由于无需另外训练分类器，因此在资源和时间方面更为高效。

5.6 Stable Diffusion

到目前为止，我们已经实现了处理 MNIST 的小型扩散模型以及小型条件扩散模型。当然，现代的扩散模型规模相当庞大，而且经过了多项改进。下面是一些进一步改进扩散模型的指引。这里我们介绍一个著名的模型——**Stable Diffusion**，它能够生成下图所示的高清图像。另外，它的代码和预训练的权重数据是公开的，任何人都可以运行这些代码。



图 5.11 Stable Diffusion 生成的高清图像